



City St George's, University of London

Course : M.Sc. Cyber Security

Project Report

**Continuous Authentication to Detect Session
Hijacking in Remote
Work Environments.**

Supervisor : Dr. Hafizul Asad

Student : Hitesh Manoj Badgujar

Date : 22 May 2026

Signed Declaration

By submitting this work, I declare that this work is entirely my own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the assessment instructions and any other relevant programme and module documentation. In submitting this work, I acknowledge that I have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. I also acknowledge that this work will be subject to a variety of checks for academic misconduct.

Signed: Hitesh Badgujar

Abstract

Remote and hybrid work often leave users logged into sensitive web applications for long periods, while many systems still rely mainly on one-time login checks. This dissertation presents a proof-of-concept continuous authentication prototype that monitors keystroke dynamics, mouse behaviour and optional face evidence during an active browser session. Public CMU-style keystroke and BALABIT-style mouse datasets are converted into window-level feature vectors and used to train behavioural models. A per-user keyboard autoencoder baseline is compared with a final SVM-based keyboard scorer. The CAE baseline shows useful separation, with an EER of about 0.164 and ROC AUC of about 0.908, but the SVM path is used in the live system because it is simpler to integrate and manage at runtime. The mouse SVM is used as a supporting behavioural signal rather than a standalone authenticator. A lightweight face module provides match and liveness scores only as optional support. The final Django monitor fuses available scores into ALLOW, STEP_UP and LOCK decisions. A labelled live test achieved 75.6% policy accuracy, with 95.5% genuine allow rate and 56.2% takeover detection, showing stable genuine use but moderate takeover detection.

Keywords: continuous authentication; behavioural biometrics; keystroke dynamics; mouse dynamics; multimodal fusion

Table of Contents

CHAPTER 1 - INTRODUCTION AND OBJECTIVE.....	6
1.1 BACKGROUND AND MOTIVATION	6
1.2 PROBLEM STATEMENT	6
1.3 PROJECT AIM AND RESEARCH QUESTIONS	7
1.4 OBJECTIVES AND SUCCESS CRITERIA	7
1.5 SCOPE AND LIMITATIONS	8
1.6 WHAT CHANGED FROM PROPOSAL TO FINAL IMPLEMENTATION	8
1.7 REPORT STRUCTURE	9
1.8 ETHICAL CONSIDERATIONS	9
1.9 USE OF GENERATIVE AI TOOLS.....	9
CHAPTER 2 - CONTEXT AND LITERATURE REVIEW	10
2.1 CONTINUOUS AUTHENTICATION AND THREAT MODEL.....	10
2.2 KEYSTROKE DYNAMICS: FEATURES, TYPICAL MODELS, AND CHALLENGES.....	10
2.3 MOUSE DYNAMICS: FEATURES, VARIABILITY, AND TYPICAL MODELS	11
2.4 FACE VERIFICATION AND LIVENESS IN CONTINUOUS SETTINGS	12
2.5 MULTIMODAL FUSION STRATEGIES AND TRADE OFFS.....	13
2.6 GAP AND HOW THIS PROTOTYPE ADDRESSES IT	13
CHAPTER 3 – METHODS.....	15
3.1 SYSTEM OVERVIEW AND DATAFLOW	15
3.2 DATASETS AND SPLITS	15
3.3 FEATURE ENGINEERING	16
3.4 MODEL DESIGNS AND TRAINING.....	18
3.4.1 Keyboard CAE baseline.....	18
3.4.2 Keyboard SVM used in the final pipeline	19
3.4.3 Mouse SVM model and training setup.....	19
3.4.4 Face scoring pipeline used for step up support	20
3.5 TRUST SCORING AND SMOOTHING IN THE LIVE SYSTEM	21
3.6 FUSION AND POLICY LOGIC (ALLOW, STEP_UP, LOCK).....	21
3.7 IMPLEMENTATION DETAILS: DJANGO ENDPOINTS, BROWSER STREAMING, AND LOGGING.....	22
3.8 EVALUATION PROTOCOL AND METRICS.....	23
CHAPTER 4 – RESULTS.....	25
4.1 OFFLINE EVALUATION METRICS SUMMARY.....	25
4.2 BEHAVIOURAL MODEL RESULTS AND DISCUSSION	25
4.2.1 Keyboard CAE verification results.....	25
4.2.2 Keyboard SVM verification and confusion metrics	28
4.3 MOUSE MODEL RESULTS	30
4.4 BEHAVIOURAL FUSION AND BEHAVIOURAL + FACE FUSION	33
4.5 TAKEOVER SIMULATION RESULT	34
4.6 REAL TIME PROTOTYPE EVIDENCE	35
CHAPTER 5 – DISCUSSION.....	37
5.1 REVISIT OBJECTIVES: WHAT WAS MET AND WHAT WAS NOT	37

5.2 KEYBOARD: WHY SVM WAS USED INSTEAD OF CAE IN THE FINAL RUNTIME	37
5.3 MOUSE: WHY IT IS WEAKER ALONE AND HOW IT STILL HELPS IN FUSION	38
5.4 FUSION AND POLICY: VALUE AND LIMITATIONS	38
5.5 THREATS TO VALIDITY AND LIMITATIONS	39
CHAPTER 6 – EVALUATION, REFLECTIONS, AND CONCLUSIONS	40
6.1 OVERALL EVALUATION AGAINST THE PROPOSAL PLAN	40
6.2 REFLECTION: WHAT I LEARNED AND WHAT I WOULD DO DIFFERENTLY	40
6.3 CONCLUSION: KEY TAKEAWAYS AND FUTURE WORK	41
REFERENCES	42
APPENDIX A – SYSTEM DIAGRAMS AND UI SCREENSHOTS	45
A.1 SYSTEM ARCHITECTURE AND WORKFLOW	45
A.2 BEHAVIOURAL TRUST MONITOR UI	47
APPENDIX B DEFINITIONS	48
B.1 KEYBOARD FEATURE VECTOR (18 FEATURES, FIXED ORDER)	48
B.2 MOUSE FEATURE VECTOR (23 FEATURES, FIXED ORDER)	49
B.2 KEYBOARD TRUST CONVERSION USING LOGISTIC SQUASHING	50
APPENDIX C MODEL CONFIGS, HYPERPARAMETERS, AND TRAINING COMMANDS	50
C.1 KEYBOARD CAE BASELINE	50
C.2 KEYBOARD SVM FINAL RUNTIME MODEL	51
C.3 MOUSE SVM MODEL	53
C.4 RUNTIME FUSION AND POLICY	54
C.5 TAKEOVER REPLAY GENERATION	55
C.6 REAL-TIME SESSION PLOTS	55
C.7 LABELLED LIVE-STREAM POLICY TEST	56
APPENDIX D –EXECUTION EVIDENCE SCREENSHOTS	57
D.1 KEYBOARD CAE TRAINING AND EVALUATION	57
D.2 KEYBOARD SVM TRAINING AND EVALUATION	59
D.3 MOUSE SVM TRAINING AND EVALUATION	60
D.4 RUNTIME PLOTS AND TAKEOVER REPLAY	61
D.5 LABELLED LIVE-STREAM POLICY EVALUATION	61
D.6 BASELINE COMPARISON EVIDENCE	62
APPENDIX E AI USE AND CODE PROVENANCE TABLE	63
APPENDIX F AI PROMPT/OUTPUT SCREENSHOTS	66
APPENDIX G – CODE AND DATA REPOSITORIES	83
G.1 PROJECT GITHUB REPOSITORY	83
G.2 CMU-STYLE KEYSTROKE DYNAMICS DATASET (KAGGLE)	83
G.3 BALABIT REMOTE DESKTOP / MOUSE DYNAMICS DATASET (KAGGLE)	83

CHAPTER 1 - INTRODUCTION AND OBJECTIVE

1.1 Background and motivation

A large part of modern work now happens inside long-running digital sessions such as email systems, cloud dashboards, banking portals, learning platforms, and internal company tools. In practice, users often keep these sessions open for hours while working from home, in labs, or in shared office spaces. After the first login, the system usually assumes that the same person remains in control until logout or timeout. In real life, that assumption is weak. A laptop may be left unlocked for a short break, a colleague may use the machine for a quick check, or a remote session may remain active while the real user is away. This creates a clear gap between “the user was authenticated once” and “the same user is still operating the session now”.

Continuous authentication tries to reduce this gap by checking identity during the session using passive signals produced during normal interaction. Behavioural biometrics such as keystroke dynamics and mouse dynamics are attractive because they can be collected during ordinary use without asking the user to stop working, although their reliability can vary with user condition, device, and task context (Teh et al., 2013; Khan et al., 2024). At the same time, continuous authentication systems must balance security with usability, because a system that reacts too often can become frustrating in practice (Baig and Eskeland, 2021; Ryu et al., 2021). This project is motivated by building a practical prototype that can monitor a live session in real time and react when the active user appears to change.

1.2 Problem statement

Session hijacking is not limited to stolen passwords at login. A more practical risk is takeover of an already authenticated session. An attacker may use an unattended workstation, a compromised browser, or remote-control malware to act as the legitimate user after the first authentication has already succeeded. In such a case, the application may continue treating later actions as genuine because the original session is still valid. Earlier work on keystroke-based impostor detection already showed that compromised passwords and shared accounts are real security problems, and that typing behaviour can help distinguish genuine users from impostors even when the password itself is known (Killourhy and Maxion, 2009).

The problem becomes serious because many digital services remain open for long periods. If a user leaves an active session running, another person may gain access to email, cloud files, financial settings, or administrative controls without needing to log in again. Continuous authentication was proposed to reduce this weakness by checking identity during the session instead of relying only on the first login event. Review work in this area shows that already logged-in services may continue running in the background, which increases the exposure if the device or session is taken over (Stylios et al., 2016). More recent work also shows that behavioural interaction features can be monitored over time and used to detect session hijacking while the session is still active (Garabato et al., 2022).

1.3 Project aim and research questions

The aim of this project is to design, build, and evaluate a working continuous authentication prototype that can monitor an active web session and react when the current behaviour no longer appears to match the legitimate user. The system focuses mainly on behavioural biometrics, especially keystroke dynamics and mouse dynamics, because these signals arise naturally during normal user interaction. Earlier work has shown that both keystroke behaviour and mouse interaction can be useful for behavioural authentication, while multimodal continuous authentication studies suggest that combining signals can improve robustness compared with relying on only one source of evidence (Teh et al., 2013; Almalki et al., 2021; Ryu et al., 2021).

This work is guided by three research questions:

1. Can keyboard and mouse windows produce a stable trust signal in a real-time loop?
2. How do threshold and smoothing choices affect the balance between usability and security?
3. Does adding an optional support signal make takeover decisions clearer, especially in borderline cases that may require a step-up response?

1.4 Objectives and success criteria

The project objectives are defined with clear success checks so that the final outcome is not only a working demo, but also something that can be evaluated in a structured way.

Objective 1 – Implement continuous feature extraction for keyboard and mouse

Success check: Capture browser events and convert them into fixed-length feature vectors at regular intervals, with a target update rate of about 150 ms, without freezing the interface or causing visible lag.

Objective 2 – Train behavioural models and convert their outputs into trust scores

Success check: Train a keyboard model and a mouse model on the prepared window datasets, and report suitable evaluation metrics such as accuracy, macro F1, FAR, and FRR, together with per-user or per-class analysis where needed.

Objective 3 – Build a fusion and policy layer with actionable outcomes

Success check: Combine the available behavioural and optional face signals into a single trust value, and map that value to three practical actions: ALLOW, STEP UP, and LOCK.

Objective 4 – Demonstrate takeover behaviour in a controlled setting

Success check: Show that when the active user changes, the trust value drops clearly and the policy moves toward STEP UP or LOCK within a short number of windows.

1.5 Scope and limitations

This work focuses on continuous authentication during an active browser session, not login-time identity proofing. The scope includes capturing keyboard and mouse events in real time, transforming them into fixed-length windows, scoring each window with trained keyboard and mouse models, and applying a simple policy with three outputs: ALLOW, STEP UP, and LOCK. The face component is treated as optional support for stronger checks rather than a full surveillance or standalone authentication system. The prototype is designed mainly for demonstration and evaluation on a single machine, and it does not claim production hardening, secure key storage, or large-scale deployment.

There are important limitations. Behavioural biometrics can vary with device, posture, fatigue, and task, which may increase false rejects over time (Teh et al., 2013; Khan et al., 2024). Some results in this dissertation also rely on controlled replay or takeover-style testing rather than a large live deployment. In addition, the face liveness component is lightweight and is not evaluated against strong anti-spoofing benchmarks, whereas recent work shows that reliable defence against replay, print, and related attacks usually needs dedicated methods and datasets (Xing et al., 2025).

1.6 What changed from proposal to final implementation

Compared to the proposal, the final work is focused more on a stable end-to-end prototype that I could run, test, and evidence clearly. In the earlier plan, I considered a per-user autoencoder as the main keyboard verification method. However, during implementation, the autoencoder path was less practical for final live scoring because it needed per-user thresholds and was harder to manage consistently in the monitor. For this reason, I changed the final keyboard runtime path to an SVM-based scorer, because it was simpler to integrate into the live monitor and easier to explain consistently. The autoencoder was kept only as a comparison baseline and not as the final runtime decision model.

The face component also changed in scope. In the proposal, it was intended to play a stronger role, but in the final implementation it became a lightweight support signal mainly for step-up situations. A full anti-spoofing evaluation was not feasible within the project scope, so I kept the face stream limited to match and basic liveness support. I also made the fusion stage more explicit by using fixed weights, simple thresholds, and rolling smoothing to reduce jitter in short windows. Finally, I improved the evidence side of the project by generating structured CSV logs, takeover-style plots, per-user breakdowns, and a browser-based Django monitor so that the final system could be demonstrated and analysed more clearly.

Some proposal components were reduced or replaced during implementation. I did not use HMMs, FaceNet, MediaPipe liveness, AES/Fernet transmission encryption, synthetic training data or a full admin dashboard in the final submitted prototype. Instead, the final system focuses on the keyboard CAE baseline, keyboard SVM runtime scorer, mouse SVM support signal, lightweight face match and liveness scores, score-level fusion, live Django monitoring, CSV logging and controlled takeover evaluation.

1.7 Report structure

Chapter 2 reviews the literature on continuous authentication and explains the main ideas behind behavioural biometrics, face support, and multimodal fusion. Chapter 3 describes the datasets, feature engineering, model design, trust scoring, fusion logic, and system implementation. Chapter 4 presents the offline results, live prototype evidence, and takeover analysis. Chapter 5 discusses the main findings, practical limitations, and the reasons behind the final design choices. Chapter 6 concludes the dissertation with overall evaluation, reflection, and future work.

1.8 Ethical considerations

This project uses behavioural and biometric-style signals, so privacy must be handled carefully. For keyboard and mouse, I used only timing and movement statistics, not the actual text typed or the content shown on screen. The face component was used only to generate a match score and a basic liveness score, and all testing was carried out in a controlled setting rather than on real employees in an organisation. In any real deployment, this type of monitoring would need to be transparent, consent-based, and supported by minimal data retention, because continuous behavioural monitoring can raise privacy concerns if it is applied without clear limits (Stylios et al., 2016; Ketab et al., 2017). Since spoofing and bias remain known risks for face systems, any face-related claims in this dissertation are limited to this prototype and should not be treated as proof of production-grade reliability (Xing et al., 2025).

1.9 Use of Generative AI tools

I used generative AI tools in this project, but I did not use them as a replacement for doing the project work myself. I used ChatGPT mainly as a support tool for explaining errors, improving code structure, checking small code blocks, debugging issues, and improving the wording of report sections. For example, I used it while working on parts such as the browser monitor flow, keyboard trust scoring, mouse SVM evaluation, fusion logic, plotting scripts, and report grammar. I also used it to help identify where some claims in the literature review were too broad, so that I could rewrite them more carefully.

All final code was checked, changed, run and tested by me before being used in the project. I also generated the final metrics, plots, screenshots and CSV outputs from my own project files. The final interpretation of results and discussion are based on my own experiments and the evidence produced by the system.

I used Napkin AI only for improving the visual layout of system diagrams. The architecture, dataflow and component meanings were defined by me.

More detailed information about where AI was used, what it helped with, what I changed afterwards, and the related repository files is provided in Appendix E: AI Use and Code Provenance.

CHAPTER 2 - CONTEXT AND LITERATURE REVIEW

2.1 Continuous Authentication and Threat Model

Traditional login checks who starts a session, but it does not prove who is still using that session later. In work settings, web and remote sessions can stay open for a long time. If a laptop is unlocked, a workstation is shared, or a remote session remains active, another person may act inside the authenticated session. In that situation, the attacker does not need to break the password again. The risk comes from the system trusting the session after the first authentication step.

Continuous authentication tries to reduce this weakness by checking the user during the session using behavioural evidence. Review work explains that continuous authentication systems can monitor user behaviour during use, instead of depending only on a single authentication event (Stylios et al., 2016; Ryu et al., 2021). In this project, the threat model is takeover during an active browser or remote-work session. The attacker may be a nearby person using an unattended device, or an external actor controlling the session remotely.

The main design issue is the balance between security and usability. If the system reacts too strongly, genuine users may be interrupted too often. If it reacts too weakly, an attacker may continue using the session for longer. My prototype therefore uses a trust score and three policy actions: ALLOW, STEP UP, and LOCK. These thresholds and smoothing rules are not copied from the literature. They are implementation choices used so the live monitor can be tested clearly.

A multimodal design is used because one signal alone is not always reliable. Keyboard behaviour can change with typing style, mouse activity can be missing when the user is reading, and face evidence can fail due to lighting or camera position. Multimodal continuous authentication work supports combining more than one signal, while also noting complexity and privacy concerns (Ryu et al., 2021; Ketab, Clarke and Dowland, 2017).

2.2 Keystroke Dynamics: Features, typical models, and challenges

Keystroke dynamics treats typing as a behavioural biometric. It uses the timing of typing rather than the actual meaning of the text. The main timing measures include key hold time and delays between key events. These timing values are often studied through digraph or latency-based features. Survey papers on keystroke biometrics describe this type of timing information and also discuss the difference between fixed-text and free-text authentication (Shanmugapriya and Padmavathi, 2009; Teh et al., 2013). Fixed-text authentication is easier to control because users type the same phrase, while free-text authentication is closer to normal use because the user types ordinary text.

Model choices in keystroke dynamics include anomaly detection, verification methods, classical machine learning models, and deep learning approaches. Killourhy and Maxion (2009) compared anomaly-detection algorithms for keystroke dynamics and showed that evaluation method and threshold choice are important when deciding whether a sample

belongs to a genuine user or an impostor. This is relevant to my project because continuous authentication does not depend only on producing a score. The score also has to be converted into a usable decision. One-class SVM has also been used for keystroke authentication where the genuine user is modelled separately, which is useful for verification-style systems (Eude and Chang, 2018). More recent work on outlier detection also shows that noisy keystroke samples can affect authentication performance, so preprocessing and careful testing are important before trusting the final output (Ismail et al., 2024).

Deep learning methods have also been explored for keystroke authentication. AbdelRaouf et al. (2023) proposed a CNN-based keystroke dynamics approach using the CMU benchmark dataset, while Xiaofeng, Shengfei and Shengwei (2019) explored CNN-RNN methods for free-text keystroke authentication. However, this project uses a compact 18-dimensional feature vector and a real-time browser loop, so a simpler SVM-based runtime model was more practical to train, test and integrate. Work on non-conventional keystroke dynamics also shows that features beyond standard timing can be useful for authentication, so this project includes correction behaviour, burst information and idle fraction as additional keyboard features (Alsultan, Warwick and Wei, 2017).

The main limitation is that typing behaviour is not fixed. It can change with the user, the keyboard, the task and the condition of the session. Because of this, a continuous authentication system should not depend on one perfect threshold. It needs careful testing, per-user analysis and a policy that can handle uncertain windows. In this project, keyboard features avoid storing typed content, but still keep timing and control-behaviour summaries. The keyboard model is therefore treated as a behavioural trust signal and then combined with mouse and optional face support.

2.3 Mouse dynamics: features, variability, and typical models

Mouse dynamics studies how a user moves and clicks the pointer during interaction with a graphical interface. It focuses on behaviour such as movement paths, timing, clicks and action patterns, not on the content selected by the user. Survey work on mouse dynamics describes the area through raw mouse attributes, feature extraction, datasets, algorithms, fusion methods, performance and limitations (Khan et al., 2024). This makes mouse behaviour a useful candidate for continuous authentication because it can be collected in the background while the user is working.

Typical mouse features are built from position and time data. These can include movement duration, pointer distance, speed, acceleration, direction change, pauses, point-and-click actions, and drag-and-drop actions. Antal and Egyed-Zsigmond (2019) used the BALABIT dataset and segmented raw mouse data into actions such as mouse move, point-and-click, and drag-and-drop before extracting features for impostor detection. This is directly relevant to my project because my mouse pipeline also converts raw movement traces into fixed window-level features before classification.

Online mouse-based continuous authentication has also been studied using clickstream data. Almalki, Assery and Roy (2021) collected online mouse-dynamics data from 20 participants, extracted approximately 87 features, and evaluated machine learning and deep learning classifiers for continuous authentication and anomaly detection. Their work supports the idea

that mouse behaviour can be used as an online behavioural signal, but it does not mean mouse alone is always enough for reliable authentication.

In my project, mouse is treated as a supporting behavioural signal. This is because mouse evidence depends on whether the user is actually moving the pointer. During reading, waiting, or keyboard-only activity, mouse information may be limited. The BALABIT work also notes that short benchmark test sessions make mouse evaluation challenging (Antal and Egyed-Zsigmond, 2019). For this reason, my final system does not use mouse as a standalone gatekeeper. Instead, the mouse SVM provides an additional trust score that is fused with keyboard trust. This is a more cautious use of mouse dynamics and matches the prototype results, where mouse helped the live trust signal but was not strong enough to act as the only LOCK decision source.

2.4 Face verification and liveness in continuous settings

Face-based checks can add useful identity evidence, but they also create privacy and spoofing risks. In a continuous authentication system, face should not be treated as the only proof of identity. It is better used as an additional signal when behaviour becomes uncertain.

Multimodal authentication work explains that systems can combine physiological traits, such as facial features, with behavioural traits, such as keystroke dynamics, instead of relying on one modality only (Ryu et al., 2021). In my project, this is why face is kept as optional support rather than the main decision model.

A practical face pipeline normally needs a way to locate the face before any matching can happen. MTCNN is relevant here because it was proposed as a cascaded CNN framework for joint face detection and alignment, with stages that produce face candidate windows, refine them, and output facial landmark positions (Zhang et al., 2016). This supports the use of a pre-trained face detection and alignment approach, but it does not prove face recognition or liveness performance by itself. In this prototype, the face stream is therefore used only to support the behavioural trust score.

The main weakness of face-based authentication is spoofing. Face anti-spoofing research discusses attacks such as print photos, replay videos, 3D masks and AI-generated faces or videos, and treats face anti-spoofing as a separate problem from basic face detection (Xing et al., 2025). The same survey also discusses datasets, deep learning methods and challenges such as generalisation to unknown attacks (Xing et al., 2025). Because of this, my lightweight liveness score should not be described as strong anti-spoofing.

There is also a privacy issue. Continuous authentication research notes that behavioural systems can collect intrusive information and that privacy needs to be protected (Stylios et al., 2016). For that reason, I frame webcam evidence cautiously. The face module provides match and liveness support for the prototype, but it is not presented as a production-grade face-authentication or anti-spoofing system.

2.5 Multimodal fusion strategies and trade offs

Multimodal continuous authentication is used because one signal alone is not reliable in every situation. Keyboard behaviour can change with typing speed, fatigue or keyboard type. Mouse evidence may be weak when the user is mainly reading or typing. Face evidence can also fail when the camera view, lighting or head position changes. A systematic review of continuous multimodal biometric authentication discusses the use of different biometric modalities together instead of relying on only one source of evidence (Ryu et al., 2021). In this project, the reason for fusion is practical: if one channel is missing or weak, another channel may still provide useful support.

Fusion can be applied at different levels. Feature-level fusion combines features from different sources into one larger feature vector before training a model. This can be useful, but it also needs the modalities to be available in a consistent way. Martín et al. (2022) specifically studied combining behavioural information at the feature level for continuous authentication, so this paper is a suitable reference for that type of fusion. In my project, I did not use feature-level fusion because keyboard, mouse and face are not always available at the same time. For example, mouse can be idle and face evidence can be missing.

Score-level fusion is more suitable for this prototype. Each model first gives its own trust score, and then the backend combines those scores using rules. This makes the system easier to debug because keyboard, mouse and face can be checked separately before they are combined. It also allows fallback behaviour. If the webcam is unavailable, the system can still use keyboard and mouse. If mouse data is missing for a short time, the system can still use keyboard and the last valid behaviour score. Decision-level fusion is simpler, because each modality gives a final decision and the system combines those decisions, but it loses more information than score-level fusion.

The main trade-off is still between security and usability. Continuous authentication has to be careful because a strict system can interrupt genuine users, while a relaxed system can miss suspicious behaviour. Security, privacy and usability are treated as important concerns in continuous authentication research (Baig and Eskeland, 2021). In this project, the exact fusion weights and thresholds are not copied from the literature. They are implementation choices in `trust_fusion.py` and `views.py`. I use score-level fusion because it is transparent, easy to test, and suitable for a proof-of-concept monitor.

2.6 Gap and how this prototype addresses it

The literature shows that behavioural biometrics can support continuous authentication, but it also shows that the problem is not solved by one signal or one accuracy number. Keystroke dynamics can provide useful timing evidence, but it is affected by user variation and threshold choice. Mouse dynamics can add another behavioural stream, but mouse activity may be limited during reading or keyboard-heavy tasks. Face evidence can support identity checking, but it brings privacy and spoofing concerns. Multimodal continuous authentication is therefore useful as a general direction, but it still needs careful implementation and evaluation (Ryu et al., 2021; Baig and Eskeland, 2021).

The gap addressed in this project is the practical connection between offline behavioural models and a live browser-based trust monitor. Offline model evaluations often focus on classification or verification results, but a continuous authentication system also needs to show how the score behaves over time, how missing evidence is handled, and when a policy should move from ALLOW to STEP UP or LOCK. This project therefore focuses on building a small but complete prototype rather than only reporting offline model scores.

The prototype addresses this gap by linking the main parts of the system into one working flow. The browser page captures keyboard and mouse activity, converts them into window-level feature vectors, and sends them to Django endpoints. The backend loads saved models, generates keyboard and mouse trust scores, applies score-level fusion, and writes live trust values to CSV logs. Face evidence is included only as optional support. The system also produces plots for trust over time, action counts and takeover replay, so the discussion can include runtime behaviour as well as offline metrics.

This chapter explains why these design choices are reasonable. Keystroke dynamics and mouse dynamics provide the behavioural basis for the system. Face evidence is included only in a limited way because proper anti-spoofing would require more dedicated evaluation. Multimodal fusion is used at score level because it is simpler to debug and can handle missing signals better than a single large combined model. The final prototype does not claim to be production-ready. Its contribution is a code-complete proof of concept that shows how keyboard, mouse and optional face evidence can be combined into one behavioural trust monitor, and what limitations appear when the system is tested with offline results, live logs and controlled takeover replay.

CHAPTER 3 – METHODS

3.1 System overview and dataflow

The implementation is packaged as a web prototype so that capture, scoring, fusion and policy decisions can be seen together in one flow. A Django backend exposes endpoints that accept keyboard, mouse and face-related inputs and return trust scores. On the client side, the monitor page listens to browser keyboard and mouse events, builds short windows, and sends the feature vectors to the backend at a cadence of about 150 ms. This keeps the prototype close to a real active session, where evidence keeps arriving and the system must respond continuously.

For keyboard input, each window is converted into an 18-dimensional vector based on timing and control behaviour. The saved keyboard SVM produces decision scores across enrolled users, and the runtime scorer converts the score margin into a keyboard trust value. For mouse input, each window becomes a 23-dimensional vector summarising movement, pauses and trajectory behaviour. A second SVM produces a mouse trust value. Face is treated separately. The webcam stream is used only to produce optional face match and lightweight liveness scores, not as the main authentication model.

Short windows can be noisy, so the backend does not treat every single tick as a final decision. The system keeps a short rolling history of fused trust values and also holds recent modality scores briefly when keyboard, mouse or face evidence is temporarily missing. This avoids sudden drops when the user pauses typing or keeps still in front of the camera.

The final fused score is mapped to three actions: **ALLOW**, **STEP UP**, and **LOCK**. These policy actions and thresholds are implementation choices in this prototype. They are used to test how a continuous trust monitor can behave in practice, while the wider literature supports the general need to balance security and usability in continuous authentication systems (Baig and Eskeland, 2021; Ryu et al., 2021). During runs, the prototype writes time-series logs of trust values and actions to CSV files so that the behaviour can be analysed later.

3.2 Datasets and splits

This project uses two public behavioural biometric datasets for offline model training and testing, and a separate live browser stream for runtime demonstration. The keyboard and mouse models were not trained from live browser activity. Instead, the live browser stream was used later in the Behavioural Trust Monitor to check how the saved models behave during an active session. Where live data is used for accuracy-style checking, it is treated separately as a labelled live test and not mixed with the offline training data.

For the keyboard channel, I used a CMU-style keystroke dynamics dataset obtained from Kaggle. The raw records were converted into fixed window-level rows, where each row represents one short keyboard segment for one user. Each row contains a `user_id` label, optional metadata such as `session_id` and `window_id`, and the final 18 engineered keyboard features used by the models. In the final implementation, keyboard modelling is based on three explicit split files: `keyboard_train_windows.csv`, `keyboard_val_windows.csv`, and

keyboard_test_windows.csv. The split contains 51 users. The training file contains 12,240 rows, the validation file contains 4,080 rows, and the test file contains 4,080 rows. The same users are present across the three files, but the windows are different. The training split is used to fit the model and scaler, the validation split is used for threshold or hyperparameter selection, and the test split is used only for final held-out evaluation.

For the mouse channel, I used the BALABIT remote desktop mouse dataset obtained from Kaggle. The raw session files were converted into window-level rows, where each row contains a user_id, a raw session or file identifier, and the 23 engineered mouse features described in Section 3.3. The final mouse pipeline uses mouse_windows_train.csv as the base tuning file, mouse_windows_train_augmented.csv as the final training file, and mouse_windows_test.csv as the held-out evaluation file. In the final run, the tuning file contained 23,358 rows from 10 users, and the held-out test file contained 23,792 rows. Grouped cross-validation was used during tuning so that windows from the same raw file did not leak across folds. After selecting the hyperparameters, the final mouse SVM was trained on mouse_windows_train_augmented.csv. In the final run this file had the same row count as the base training file, so I do not claim that augmentation increased the dataset size.

For the face channel, I did not use a separate offline training dataset. Face is handled only at runtime through webcam frames captured in the browser. These frames are used to enrol a simple template and generate a face match score and a lightweight liveness score. The face component is therefore treated as an optional support signal in the live prototype, not as a fully benchmarked standalone biometric model.

This split design keeps the offline training and testing path separate from the live runtime demonstration. In simple terms, keyboard and mouse models are trained and evaluated offline using prepared dataset windows, while the browser stream is used later to show how the saved models behave during an active session. This avoids mixing training data, held-out evaluation data, and live runtime evidence.

3.3 Feature Engineering

3.3.1 Keyboard 18 dimensional feature vector

For keyboard dynamics, I did not use the actual text typed by the user. I only used timing and control behaviour. This is important because a continuous authentication system should avoid storing typed content where possible. The aim is to learn the user's interaction rhythm, not the meaning of what they write. Keystroke dynamics literature commonly studies timing information based on key press and key release events, including hold time and latency between successive key events (Shanmugapriya and Padmavathi, 2009; Teh et al., 2013). In my browser implementation, the equivalent events are captured using JavaScript keydown and keyup events. The exact 18-dimensional vector used in this project is my own implementation choice based on these timing ideas and the needs of the live prototype.

From the browser stream, I build short keyboard windows at roughly the same rate as the runtime loop. Each window is summarised into a fixed 18-dimensional vector. This keeps the model input consistent between training, offline testing and real-time scoring. The vector is built from two groups of information.

The first group captures timing. I compute dwell time as the gap between keydown and keyup for the same key. I also compute inter-key latencies, including the gap from one key press to the next key press and the gap from a key release to the next key press. Inside each window, I use summary statistics such as mean, standard deviation and percentiles. This keeps the feature vector compact while still representing the user's typing rhythm and pauses.

The second group captures control behaviour. I include features such as backspace rate, burst behaviour and idle fraction. These are not meant to identify the actual text. They describe how the user interacts with the keyboard, for example whether they type in short bursts, pause often, or make corrections. I avoid storing key identities, so the same feature definition can be used across different text.

I chose 18 features mainly for practicality. They are light enough to compute in JavaScript, simple enough to keep in a fixed order, and suitable for SVM and CAE testing. This fixed vector also avoids the extra training and deployment complexity of sequence models, which would need more data and longer context.

3.3.2 Mouse window features

Mouse behaviour is captured as a stream of pointer positions with timestamps. In the browser client, the page records mousemove and pointermove events and builds mouse features during the runtime loop. Each build uses the recent pointer points available in the client window. If there are too few points, the function returns no mouse feature vector for that tick. This matters because mouse evidence is not always available during normal use, for example when the user is mainly reading or typing. Mouse dynamics literature supports using movement and click behaviour as behavioural biometric evidence, but the exact 23-feature vector used here is my own implementation choice (Khan et al., 2024; Antal and Egyed-Zsigmond, 2019).

The first group of mouse features describes geometry. I calculate total path length by summing the distance between consecutive pointer points. I also calculate the straight-line distance from the first point to the last point and use it to compute straightness. I keep net displacement in x and y, their absolute values, and the bounding box width, height and area of the movement inside the window.

The second group captures movement dynamics. For each segment, I compute speed as distance over time. From speed, I keep the mean, 95th percentile and maximum. I then calculate acceleration from speed changes and jerk from acceleration changes. For acceleration and jerk, I again keep mean, 95th percentile and maximum. I also count direction-change behaviour using sign changes in movement direction.

The final part covers pause behaviour. I include a pause-ratio feature based on time gaps between pointer points. Together, these 23 values summarise geometry, speed, acceleration, jerk, displacement, direction changes and pauses. This fixed vector keeps the mouse model lightweight and consistent between offline training and live scoring.

3.3.3 Face features used in the prototypes

Face is used only as an additional support signal in this prototype. It is not the main authentication model, and it is not trained offline like the keyboard and mouse models. The purpose of the face stream is to provide extra confidence during uncertain cases, mainly when behavioural trust is close to the STEP UP region.

In the browser monitor, webcam frames are captured at a slower rate than keyboard and mouse events. The frame is sent to the backend as an image string through the face scoring endpoint. On the backend, the system first checks whether a face is present. If no face is detected, the API returns a no-face response and the fusion layer treats face evidence as missing rather than automatically suspicious. This is important because a user may move slightly away from the camera or the frame may temporarily fail.

When a face is detected, the system extracts a simple face representation and compares it with the enrolled face template. This produces a face match score between 0 and 1. Along with the match score, the system calculates a lightweight liveness score using short-term changes in face appearance and motion. These two values are the face features used by the fusion logic.

The face component is intentionally limited. It is not a full face-recognition benchmark and it is not a strong anti-spoofing system. Basic motion-based liveness can still be weak against printed photos, replayed videos or more advanced attacks. Face anti-spoofing work shows that stronger protection normally needs dedicated large datasets, trained anti-spoofing models and proper evaluation protocols (Xing et al., 2025). Because of this, I treat face only as optional support for trust fusion, not as a standalone guarantee of identity.

3.4 Model designs and training

3.4.1 Keyboard CAE baseline

For the keyboard channel, I tested two model families using the same 18-dimensional feature vector from Section 3.3. The first was a per-user autoencoder baseline, implemented through `train_kb_cae_cmu.py` and evaluated using `eval_kb_cae.py`. The idea was to train one model for each user using only that user's genuine keyboard windows. This fits a verification-style view of authentication, where the model learns normal behaviour for one claimed user and treats larger deviations as more suspicious. One-class and verification-style approaches are also discussed in keystroke authentication work, especially when the aim is to model the genuine user separately (Eude and Chang, 2018).

In the final pipeline, CAE training uses the explicit keyboard split files. For each user, the script fits a `StandardScaler` on that user's genuine training windows only. It then trains an encoder-decoder network to reconstruct the 18-dimensional input vector. The model uses dense layers with a 16-unit bottleneck and mean squared reconstruction loss. The expectation is that genuine windows should reconstruct with a lower residual, while impostor windows should usually produce a larger residual.

Evaluation is done using validation and test splits separately. For each user, the validation split is used to choose the reconstruction-error threshold τ . The model then uses that fixed

threshold on the held-out test split. In the test stage, windows from the claimed user are treated as genuine and windows from other users are treated as impostor. The evaluation records FAR, FRR, accuracy and ERR per user, and also writes macro summaries and score files for ROC, PR and EER plots.

The CAE baseline showed useful separation after this corrected evaluation, with EER around 0.164 and ROC AUC around 0.908. However, I kept it as a baseline rather than the final runtime model because it needs one model and one threshold per user. The SVM path was simpler to integrate into the live monitor and easier to manage as one global runtime scorer.

3.4.2 Keyboard SVM used in the final pipeline

The second keyboard model is an SVM-based classifier trained once across all users. This is implemented in `eval_kb_svm.py` and is the keyboard model used in the final live runtime path. I moved to this approach because it was easier to integrate into the monitor than the per-user CAE baseline. Keystroke dynamics literature supports using timing-based behavioural features for authentication, but the final SVM design and hyperparameter choices in this project are based on my own implementation and testing results (Teh et al., 2013; Killourhy and Maxion, 2009).

The script uses the explicit keyboard split files: `keyboard_train_windows.csv`, `keyboard_val_windows.csv`, and `keyboard_test_windows.csv`. The split contains 51 users. The training file has 12,240 rows, the validation file has 4,080 rows, and the held-out test file has 4,080 rows. A global `StandardScaler` is fitted on the training windows only and reused for validation, testing and runtime scoring. The classifier is `sklearn.svm.SVC` with an RBF kernel and `class_weight="balanced"`.

Hyperparameters are chosen using a small validation grid. The tested values include different C and γ combinations, and the best validation result was obtained with $C = 20.0$ and $\gamma = 0.1$. After selecting these values, the final SVM is retrained on the combined train and validation windows, giving 16,320 rows for final fitting. The held-out test split is then used only for final evaluation.

For verification-style reporting, each user's test windows are treated as genuine and all other users' test windows are treated as impostor. The script records FAR, FRR and accuracy per user, and macro summaries are saved to `artifacts/kb_svm_eval_metrics.csv` and `artifacts/kb_svm_eval_overview.json`. The trained model and scaler are saved under `Models/kb_svm/` so the Behavioural Trust Monitor uses the same normalisation and decision boundary during live scoring.

3.4.3 Mouse SVM model and training setup

For the mouse channel, I used an SVM because the input is already a compact 23-dimensional window vector and the live monitor needs fast scoring. Mouse dynamics literature supports the use of movement and click behaviour as behavioural biometric evidence, but mouse behaviour is also affected by the amount and type of interaction available in a session (Khan et al., 2024; Almalki, Assery and Roy, 2021). For that reason, I kept the mouse model simple and treated it as a supporting behavioural signal rather than the main authentication gate.

Training is done using `Apps/behavior/train_mouse_svm.py`. The script first loads `Data/mouse_windows_train.csv` for hyperparameter tuning. In the final run, this tuning file contained 23,358 windows from 10 users and 65 raw files. Users with too few samples are filtered using `MIN_PER_USER = 40`. The features are taken in the fixed 23-feature order described in Section 3.3.2. A `StandardScaler` is fitted on the training windows and saved for later runtime use.

Hyperparameter selection uses grouped cross-validation by raw file. This is important because windows from the same raw mouse recording can be very similar. Grouping by file reduces the risk of testing on windows that are too close to the training windows. The model is an RBF-kernel `sklearn.svm.SVC` with `class_weight="balanced"`. The manual grid tested values of `C` and `gamma`, and the best setting in the final run was `C = 50.0` and `gamma = 0.1`, with grouped CV mean accuracy of 0.3256 and grouped CV mean macro F1 of 0.2670.

After tuning, the final mouse SVM is trained on `Data/mouse_windows_train_augmented.csv` with `probability=True`, so that the live runtime can use `predict_proba`. In the final run, this augmented file had the same row count as the base training file, so I do not claim that augmentation increased the training set size. The trained artefacts are saved under `Models/mouse/` as `mouse_svm.joblib`, `mouse_scaler.joblib`, and `mouse_meta.json`. Final held-out evaluation is done separately by `Apps/behavior/eval_mouse.py` on `Data/mouse_windows_test.csv`, which contains 23,792 windows. Mouse intrusion detection work supports using mouse dynamics as part of a security signal, but in this project I use it only as support in fusion, not as a standalone lock decision (Antal and Egyed-Zsigmond, 2019).

3.4.4 Face scoring pipeline used for step up support

For the face channel, I did not train a separate face classifier like the keyboard and mouse SVMs. The final implementation uses a lightweight runtime scoring pipeline that can support STEP UP decisions when behavioural trust becomes uncertain. I kept it limited because a full face-authentication and anti-spoofing system would need dedicated face datasets, stronger models and proper benchmark testing, which were outside the scope of this project. Face is therefore treated as optional support, not as the main authentication path.

In the runtime flow, the browser captures a webcam frame at intervals and sends it to the backend face endpoint. The backend first checks whether a face is present. A proper face pipeline normally needs face detection and alignment before matching. MTCNN is a well-known example of this type of detection and alignment approach, because it jointly performs face detection and facial landmark localisation (Zhang et al., 2016). In my final prototype, the face module is kept lightweight and uses a simple detected face region and enrolled template comparison to produce a match score between 0 and 1.

Along with the match score, the system computes a basic liveness score from short-term visual change and motion cues. This is only a lightweight heuristic. It is not presented as a strong anti-spoofing method. Face anti-spoofing research discusses attacks such as printed photos and replay videos and shows that reliable defence normally needs dedicated anti-spoofing models, datasets and evaluation protocols (Xing et al., 2025). Because of this, I use

the face score only as support in the fusion stage. It can improve confidence when available, but the final system can still run in behaviour-only mode without webcam evidence.

3.5 Trust scoring and smoothing in the live system

Raw model outputs are not used directly as final decisions because short windows can be noisy. A genuine user can produce one weak keyboard or mouse window because of a pause, correction, or temporary lack of movement. In the live pipeline, I convert outputs into trust scores between 0 and 1 and then apply short-term holding and smoothing before producing a policy action. The general reason is the security and usability trade-off in continuous authentication, but the exact holding rules and thresholds are my own implementation choices (Baig and Eskeland, 2021; Ryu et al., 2021).

Keyboard trust is computed in `Apps/behavior/ae_conditional.py` inside `RuntimeScorer`. The keyboard SVM produces class decision scores. I use the separation between the top two scores and convert that margin into trust using a logistic squashing function. I used this because plain maximum probability looked too flat during live testing. When there is no fresh typing, the frontend does not send a fake zero keyboard score. The backend briefly holds the last valid keyboard trust instead.

Mouse trust is computed in `Apps/behavior/views.py` using the saved mouse SVM, scaler and fixed 23-feature order. The runtime scales the mouse vector, calls `predict_proba`, and converts the maximum class probability into mouse trust. If there is no movement in a tick, mouse is treated as missing evidence, not as failure. The backend briefly holds recent mouse trust because mouse evidence depends on active pointer movement.

Face trust uses face match and liveness. Match comes from comparison with the enrolled template. Liveness is a lightweight motion-based cue, so it is kept as support only. The current fusion uses 0.9 match and 0.1 liveness. This is not claimed as strong anti-spoofing, because proper anti-spoofing needs dedicated methods, evaluation protocols and larger datasets (Xing et al., 2025).

Fusion and policy run in `Apps/behavior/trust_fusion.py` and `Apps/behavior/views.py`. Behaviour trust uses 0.8 keyboard and 0.2 mouse, with a clamp so it cannot fall below keyboard trust. Overall trust uses behaviour and optional face support, but face cannot drag behaviour trust down. The current thresholds are: below 0.30 is LOCK, 0.30 to below 0.55 is STEP_UP, and 0.55 or above is ALLOW.

3.6 Fusion and policy logic (ALLOW, STEP_UP, LOCK)

Fusion is done at score level because each modality already produces its own trust score. This is simpler than feature-level fusion in this prototype because keyboard, mouse and face evidence are not always available at the same time. For example, mouse can be idle when the user is reading, and face can be unavailable if the camera frame fails. Literature supports the general idea of combining multiple modalities in continuous authentication, but the exact

score-level weights used here are my implementation choices, not formulas copied from papers (Ryu et al., 2021).

In the code, fusion is implemented in Apps/behavior/trust_fusion.py and applied by the fuse_scores endpoint in Apps/behavior/views.py. First, keyboard and mouse are combined into behavioural trust: **$\text{behaviour_trust} = 0.8 * \text{kb_trust} + 0.2 * \text{mouse_trust}$**

Keyboard is given the higher weight because, in my offline and live testing, it was the stronger behavioural signal for impostor rejection. I then apply a clamp rule:

$\text{behaviour_trust} = \max(\text{behaviour_trust}, \text{kb_trust})$

This rule was added after testing because mouse trust sometimes dropped during sparse movement. I did not want weak mouse evidence to reduce behavioural trust when keyboard evidence was stable.

Face trust is computed separately: **$\text{face_trust} = 0.9 * \text{face_match} + 0.1 * \text{face_liveness}$**

The match score has the higher weight because the liveness check is lightweight and motion-based. Face is therefore used as support, not as the main decision source.

When face is available, overall trust is calculated as:

$\text{raw_overall_trust} = 0.6 * \text{behaviour_trust} + 0.4 * \text{face_trust}$

$\text{overall_trust} = \max(\text{behaviour_trust}, \text{raw_overall_trust})$

This means face can support the behavioural score, but it cannot drag the score below behaviour trust when face motion is temporarily weak. If face is missing, the system falls back to behavioural trust only.

Finally, the policy maps trust to three actions:

$\text{overall_trust} < 0.30 \quad \rightarrow \text{LOCK}$

$0.30 \leq \text{overall_trust} < 0.55 \rightarrow \text{STEP_UP}$

$\text{overall_trust} \geq 0.55 \quad \rightarrow \text{ALLOW}$

These thresholds are prototype settings from the code. They are used to test whether the system can separate normal, uncertain and suspicious states in a clear way. The literature supports the general security-usability trade-off, but the exact policy thresholds come from my implementation and testing, not from the cited papers.

3.7 Implementation details: Django endpoints, browser streaming, and logging

The system is implemented as a Django client-server prototype. The browser is responsible for capturing user activity, while the Django backend is responsible for model loading, scoring, fusion and logging. The main user interface is templates/behavior/monitor.html. This page listens to keyboard, mouse and webcam activity, builds feature windows where needed, and sends JSON requests to the backend. I used standard browser APIs such as keyboard

events, pointer or mouse events, webcam frame capture and fetch() requests. These are standard web development functions, not reused authentication code.

On the server side, the main logic is in `Apps/behavior/views.py`. The browser calls the `/behavior/stream_keystrokes` URL endpoint, which receives keyboard and mouse feature vectors and returns per-modality scores. The `/behavior/fuse_scores` endpoint combines the available scores into behavioural trust, face trust and the final policy action. The `/behavior/face_enroll` and `/behavior/face_score` endpoints handle the optional face template and runtime face score. Keyboard scoring uses the saved scaler and SVM model under `Models/kb_svm/`. Mouse scoring uses the saved scaler, SVM model and metadata under `Models/mouse/`.

During development, I tested the parts separately before combining them. I first checked whether the browser was sending feature vectors correctly. Then I checked whether the backend returned valid JSON and whether feature dimensions matched the saved scalars. After that, I tested the full loop in the monitor page. I also added session IDs and short hold/decay behaviour because early live runs dropped trust too quickly when the user paused typing or stopped moving.

For evidence, each fusion tick is logged to `artifacts/realtime/live_trust_timeseries.csv`. The log stores timestamp, keyboard trust, mouse trust, behavioural trust, face trust, fused trust and final action. These logs are used in Chapter 4 to generate time-series plots, trust summaries and policy action counts. This makes the web prototype auditable instead of only being a visual demo.

3.8 Evaluation protocol and metrics

I evaluated the system in three ways. First, I used offline evaluation on prepared keyboard and mouse window datasets. Second, I used controlled takeover replay to show how trust changes when the active user switches. Third, I used a small labelled live-stream test to check how the saved system behaves on unseen browser activity.

For offline keyboard evaluation, I report both the CAE baseline and the final keyboard SVM. The CAE is evaluated as a per-user verification model. Its validation split is used to choose the reconstruction threshold, and the held-out test split is used to report FAR, FRR, accuracy, ERR, EER, ROC AUC and PR area. The keyboard SVM is evaluated on `keyboard_test_windows.csv` in verification style, where each user's windows are treated as genuine and all other users' windows are treated as impostor. The main SVM outputs are saved in `artifacts/kb_svm_eval_metrics.csv` and `artifacts/kb_svm_eval_overview.json`, with extra plots under `artifacts/keyboard/`.

For offline mouse evaluation, I use `mouse_windows_test.csv` as the held-out file. Each user is again tested as genuine against impostor windows from the other users. I report FAR, FRR and verification-style accuracy per user, with macro summaries saved in `artifacts/mouse_svm_eval_metrics.csv` and `artifacts/mouse_svm_eval_overview.json`. I also generate ROC, PR, FAR/FRR, histogram and confusion-matrix plots under `artifacts/mouse/`.

Mouse is reported as a support signal because the held-out results show it is not strong enough to act as the only authentication gate.

For runtime behaviour, I use two tests. The first is `make_takeover_plot.py`, which creates `artifacts/realtime/takeover_sim_kb.png` and its CSV. This replay uses enrolled-user keyboard trust, not generic maximum confidence, so a user switch becomes visible. The second is a labelled live-stream test using separate genuine and takeover recordings. In that test, `ALLOW` is counted as correct during genuine use, while `STEP_UP` or `LOCK` is counted as correct during takeover. This gives live policy accuracy, genuine false-alarm rate and takeover detection rate.

CHAPTER 4 – RESULTS

4.1 Offline evaluation metrics summary.

This chapter reports the results from the final trained models and the saved artefacts in artifacts/. The evaluation is done on window-level CSV files. Keyboard uses the CMU-style keystroke windows built from the Kaggle dataset, and mouse uses the BALABIT remote desktop mouse windows. The same feature order and saved scalers are reused between offline testing and the live scoring path, so the reported numbers match the final implementation.

I start with a compact summary table so the overall pattern is clear before showing per-user results and plots.

Model	FAR	FRR	Verification ACC	Main note
Keyboard CAE baseline	0.153	0.176	0.836	Useful baseline, but needs one threshold per user
Keyboard SVM final	0.008	0.400	0.984	Very low FAR, but strict for genuine users
Mouse SVM	0.084	0.750	0.583	Weak alone, useful only as support

Table 4.1 – Global summary

Source files:

- artifacts/kb_cae_metrics_explicit.csv
- artifacts/kb_svm_eval_overview.json
- artifacts/mouse_svm_eval_overview.json

The main takeaway is that no single channel is perfect. The CAE baseline separates genuine and impostor windows reasonably, but it is more difficult to manage at runtime because it needs per-user thresholds. The keyboard SVM gives the strongest impostor rejection, but its FRR is high. The mouse SVM is weaker alone, so it is used only as a support signal in fusion.

4.2 Behavioural model results and discussion

This section presents the keyboard results first because keyboard is the main behavioural stream in the final live monitor. I report the CAE baseline first, then the SVM model used in the runtime pipeline. Mouse results are presented separately in Section 4.3.

4.2.1 Keyboard CAE verification results

The CAE was trained as a per-user baseline. Each user has a separate autoencoder trained on genuine training windows only. The validation split is used to choose the reconstruction-error threshold τ , and the held-out test split is then used for final reporting. This avoids choosing the threshold on the same data used for the final test.

The corrected CAE evaluation produced a more balanced result than the earlier trial. The macro FAR was approximately **0.153**, the macro FRR was approximately **0.176**, and the macro accuracy was approximately **0.836**. The EER curve gives an EER of about **0.1641**, with ROC AUC about **0.9079** and PR area about **0.9025**. These values show that the CAE learned useful user-specific reconstruction patterns.

However, I still kept CAE as a baseline rather than the final runtime scorer. The main reason is operational complexity. It needs one model and one threshold per user, while the SVM runtime path uses one global model and scaler. This made the SVM easier to integrate into the live monitor.

Source files:

- artifacts/kb_cae_metrics_explicit.csv
- artifacts/kb_cae_overview_explicit.json
- artifacts/plots/kb_cae_scores.csv

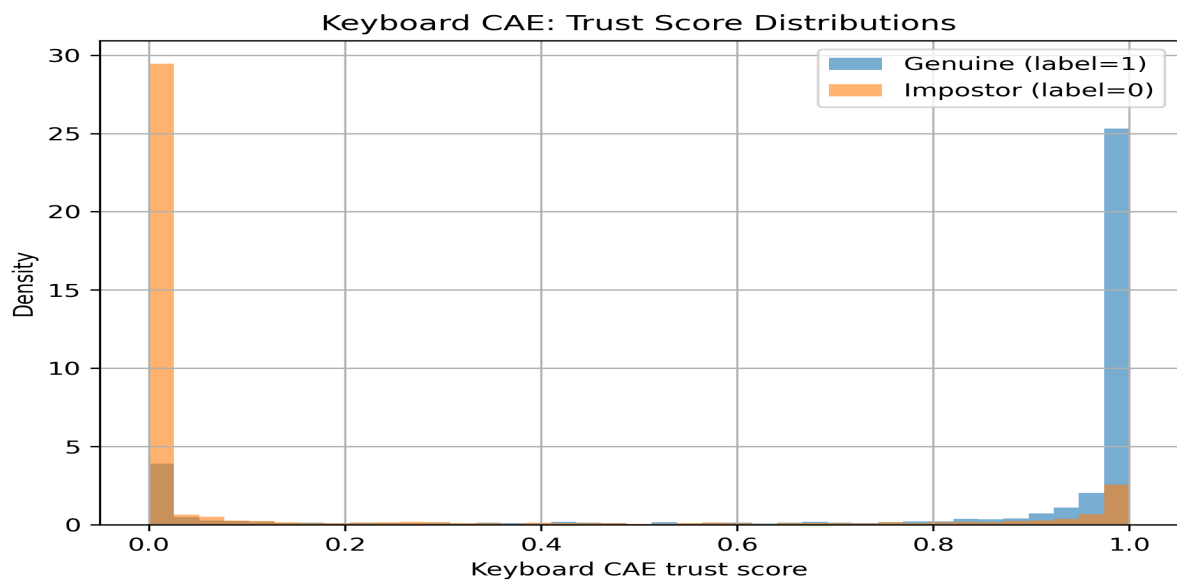


Figure 4.1 – CAE trust histograms from artifacts/plots/kb_cae_trust_histograms.png

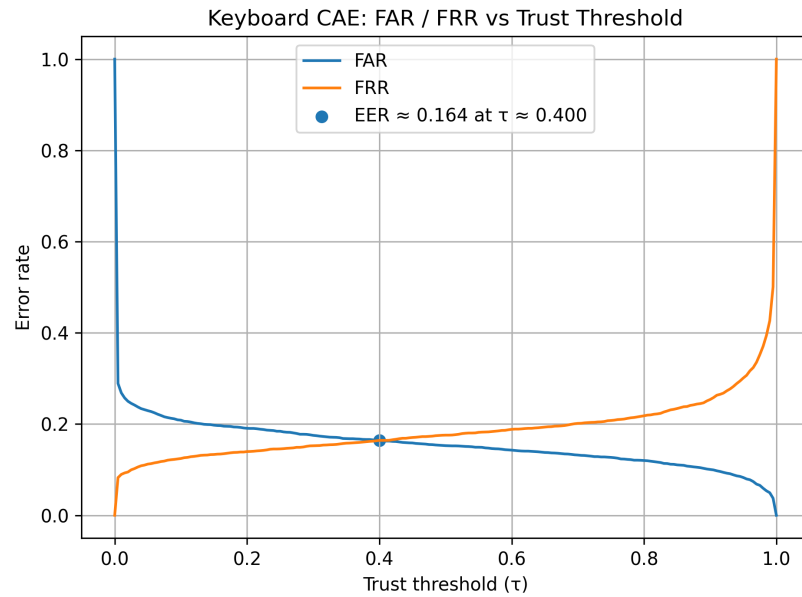


Figure 4.2 – CAE FAR and FRR curve from [artifacts/plots/kb_cae_far_frr_vs_threshold.png](#)

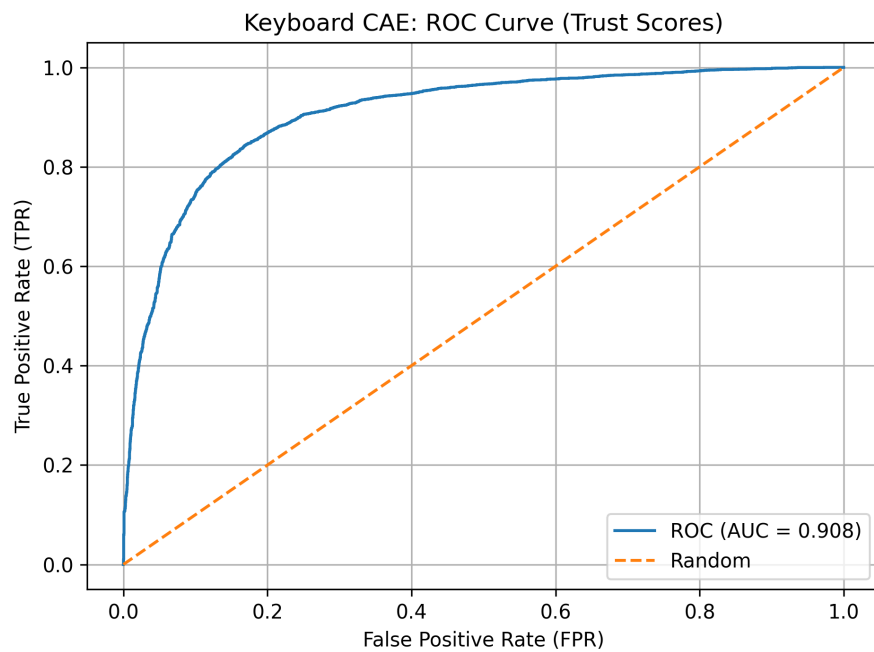


Figure 4.3 – CAE ROC curve from [artifacts/plots/kb_cae_roc_curve.png](#)

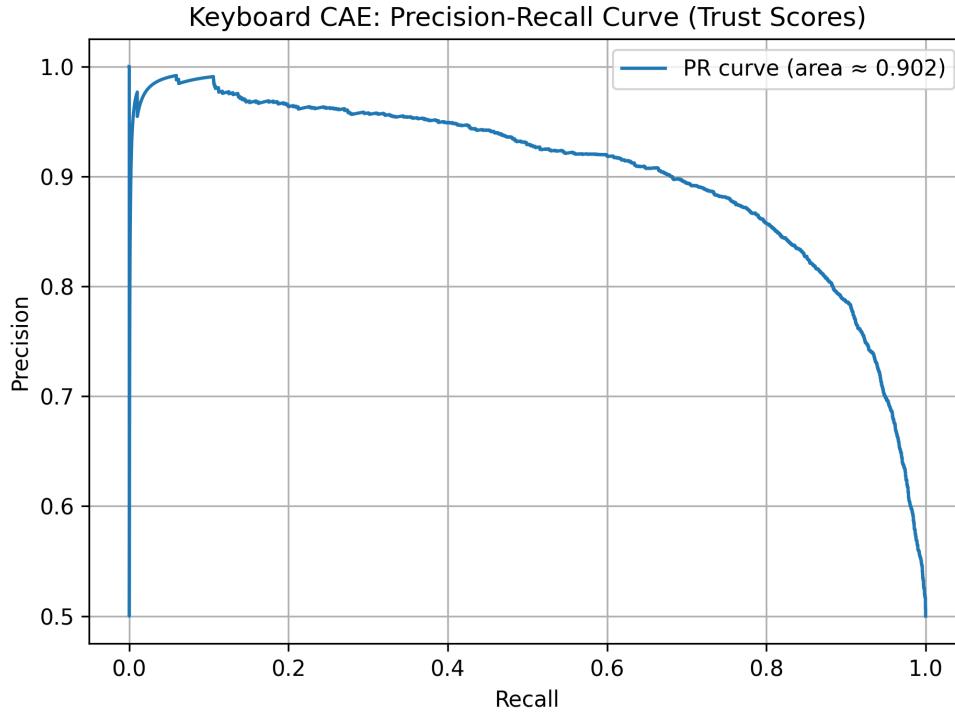


Figure 4.4 – CAE PR curve from *artifacts/plots/kb_cae_pr_curve.png*

Metric	Value
Macro FAR	0.153
Macro FRR	0.176
Macro ACC	0.836
EER	0.1641
ROC AUC	0.9079
PR area	0.9025

Table 4.2 – Keyboard CAE baseline summary

4.2.2 Keyboard SVM verification and confusion metrics

The final keyboard module is the multi-class SVM. I report it in verification style because the live system is interested in whether the current behaviour still matches the claimed user, not only which class has the highest prediction. In the final held-out test, each user has 80 genuine windows and 4,000 impostor windows from the other users.

The macro verification result was **FAR = 0.008**, **FRR = 0.400**, and **ACC = 0.984**. This shows the main trade-off clearly. The SVM is strong at rejecting impostors, because the false acceptance rate is very low. However, the false rejection rate is high, which means many genuine windows are also treated as weak or suspicious. Therefore, keyboard alone is too strict for a final live decision, but it is useful as the main behavioural signal inside fusion.

Metric	Value
Macro FAR	0.008
Macro FRR	0.400
Macro ACC	0.984
Genuine windows per user	80
Impostor windows per user	4000

Table 4.3 – Keyboard SVM verification summary

Source files:

- artifacts/kb_svm_eval_metrics.csv
- artifacts/kb_svm_eval_overview.json
- artifacts/keyboard/kb_svm_metrics.csv

The confusion-matrix plot and PR curve are included to show the operating behaviour visually. The high accuracy should not be read alone because the verification setup has many more impostor than genuine windows.

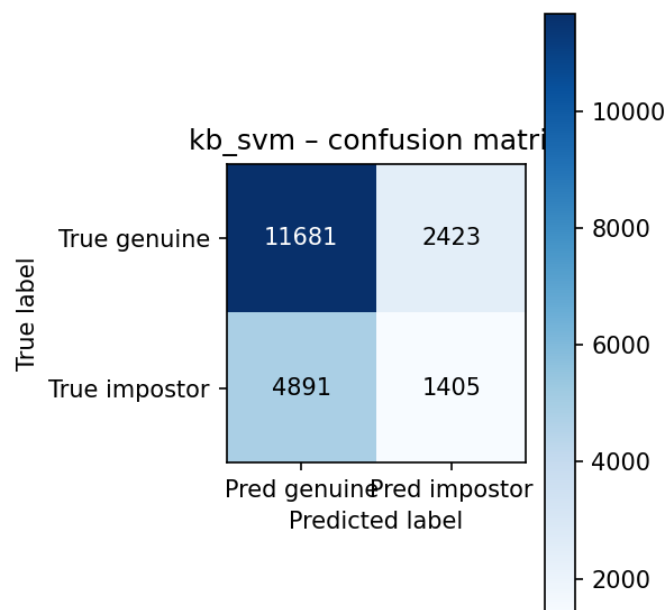


Figure 4.5 – Keyboard SVM confusion matrix from artifacts/keyboard/kb_svm_confusion.png

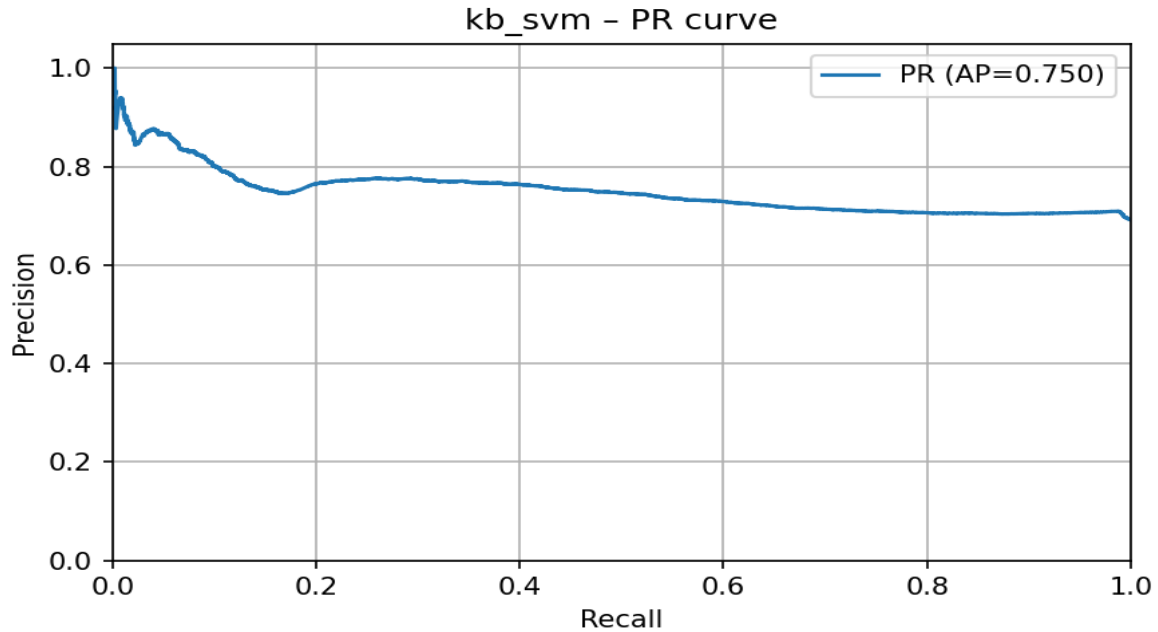


Figure 4.6 – Keyboard SVM PR curve from artifacts/keyboard/kb_svm_pr.png

4.3 Mouse model results

The mouse channel uses a single multi-class SVM trained on the 23 window features described in Chapter 3. The final evaluation is done on mouse_windows_test.csv. I report the mouse model in verification style, where each user is treated as genuine in turn and the other users are treated as impostors. This matches the way mouse trust is used in the live system: it is a supporting signal, not a standalone identity decision.

The held-out mouse evaluation shows that the mouse model is weaker than keyboard. The macro FAR is approximately 0.084, the macro FRR is approximately 0.750, and the macro verification-style accuracy is approximately 0.583. This means the model rejects many genuine mouse windows, even though the false acceptance rate is lower. In practice, this is not good enough for a direct LOCK decision. It is more useful as an extra behavioural signal that can support keyboard trust.

Metric	Value
Macro FAR	0.084
Macro FRR	0.750
Macro ACC	0.583
Test windows	23,792
Users	10

Table 4.4 – Mouse SVM held-out verification summary

Source files:

- artifacts/mouse_svm_eval_metrics.csv
- artifacts/mouse_svm_eval_overview.json
- artifacts/mouse/mouse_scores.csv

For visual evidence, I include the standard mouse plots from the artifacts/mouse/ folder.

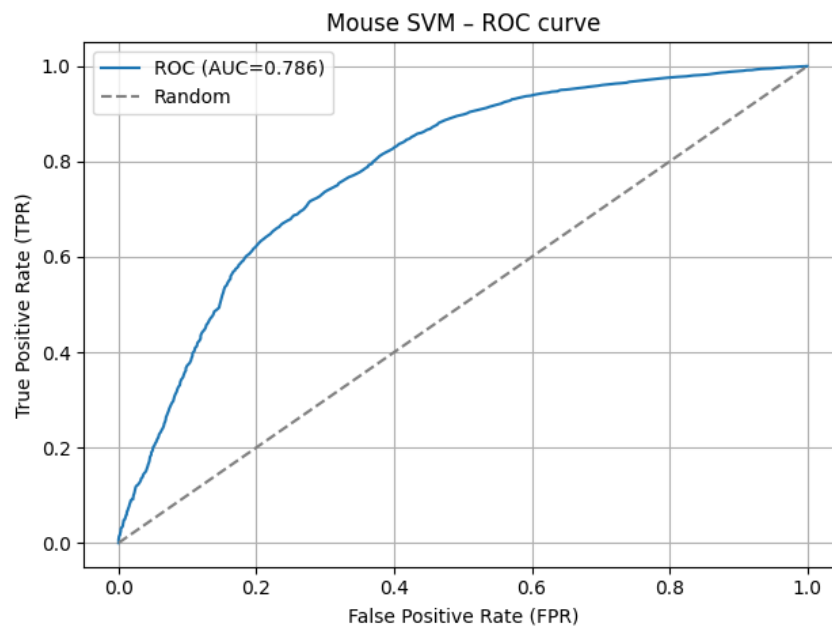


Figure 4.7 – Mouse SVM ROC curve from artifacts/mouse/mouse_roc.png

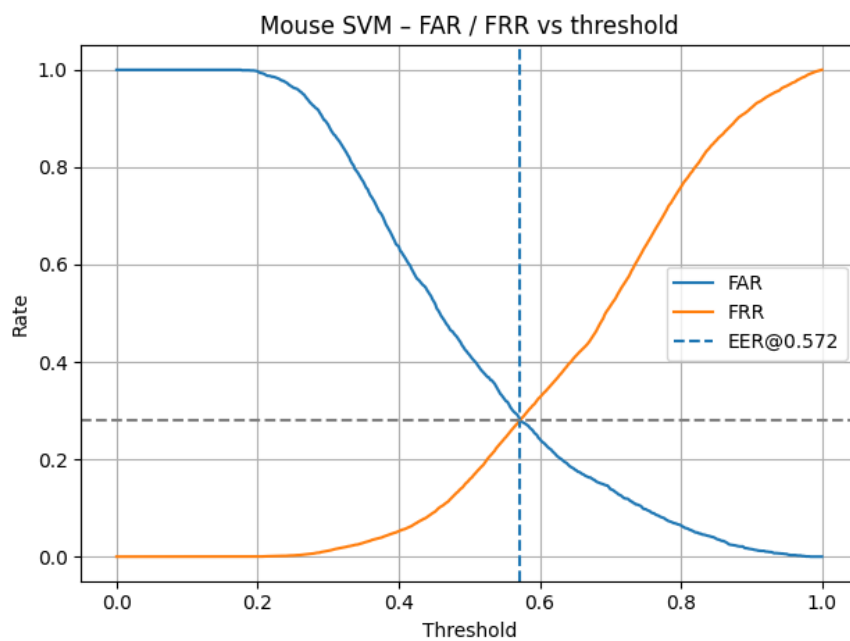


Figure 4.8 – Mouse SVM FAR and FRR curve from artifacts/mouse/mouse_far_frr.png

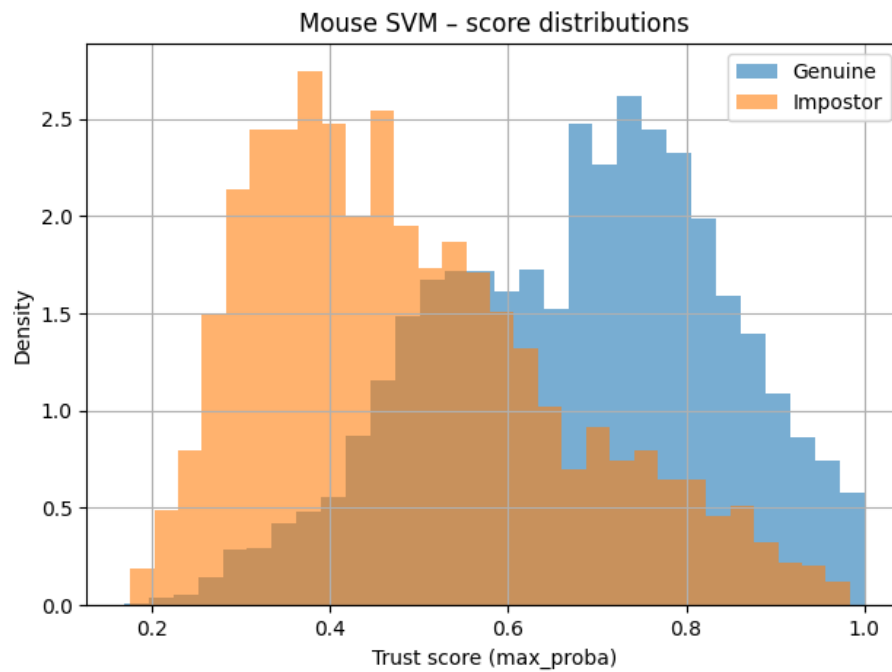


Figure 4.9 – Mouse SVM score histograms from artifacts/mouse/mouse_hist.png

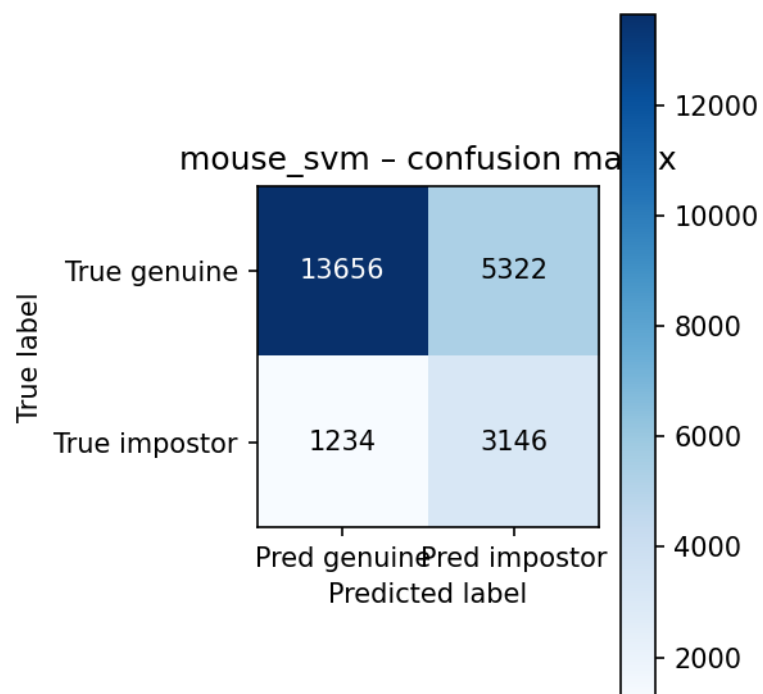


Figure 4.10 – Mouse SVM confusion matrix from artifacts/mouse/mouse_svm_confusion.png

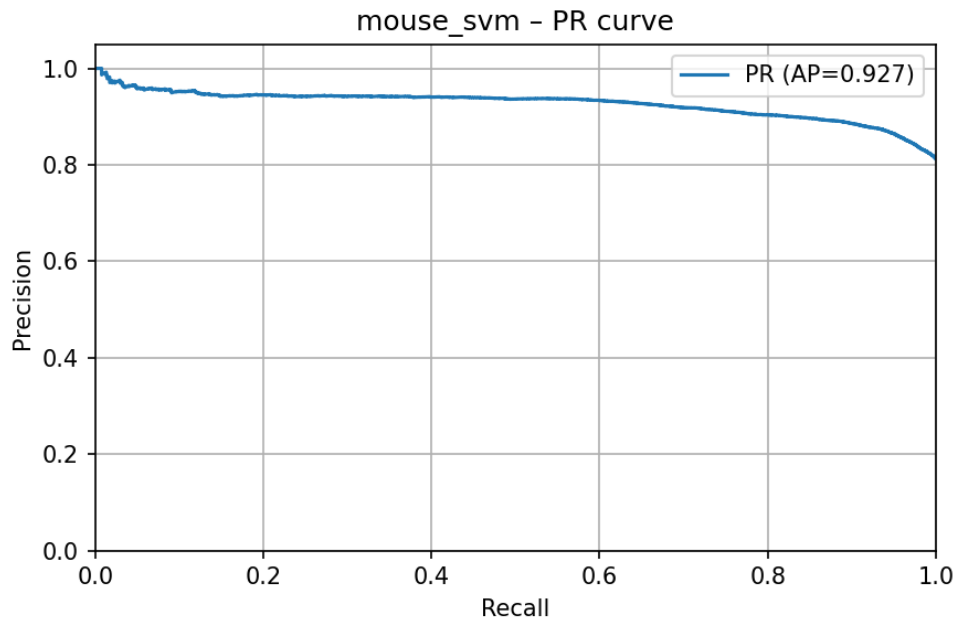


Figure 4.11 – Mouse SVM precision-recall curve from artifacts/mouse/mouse_svm_pr.png

Together, these plots show visible overlap between genuine and impostor mouse scores. This is why I treat mouse as support in fusion, not as the only authentication gate.

4.4 Behavioural fusion and behavioural + face fusion

After scoring keyboard and mouse separately, the system combines them into behavioural trust. Face is then used only as optional support when available. The aim of fusion is not to make every signal perfect. It is to reduce the effect of one weak or missing stream during a short window. To show this behaviour in practice, I summarise the recorded live-session trust values instead of only giving the formulas.

Signal	Mean	Std	Min	Max
Keyboard	0.4577	0.1992	0.0000	0.9863
Mouse	0.7347	0.0000	0.7347	0.7347
Behavioural	0.5156	0.1651	0.1469	0.9863
Face	0.8691	0.1355	0.0000	0.9214
Fused	0.6411	0.1141	0.1469	0.7119

Table 4.5 – Runtime trust summary

This table shows that mouse and face were relatively stable in this recorded session, while keyboard was more variable. The fused score stayed in a more usable range than the weaker individual windows, which supports the decision to use score-level fusion. The face value was high when available, but it is still treated only as support because the liveness check is lightweight.

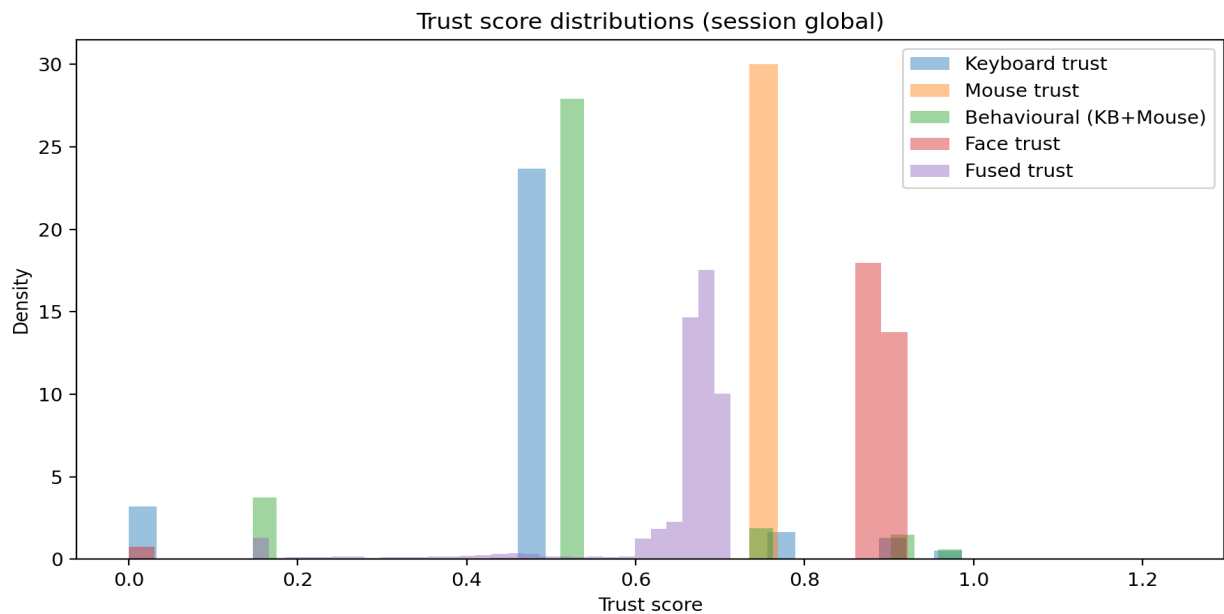


Figure 4.12 – Histograms of keyboard, mouse, behavioural, face and fused trust scores from *artifacts/realtime/fig_trust_histograms.png*

The distribution plot supports the same point. The channels do not behave equally, so the policy should not depend on one signal alone.

4.5 Takeover simulation result

To show takeover behaviour clearly, I used a controlled keyboard replay generated by `make_takeover_plot.py`. The replay uses held-out keyboard test windows. It starts with one claimed user and then switches to another user at a known takeover point. The output is saved as *artifacts/realtime/takeover_sim_kb.png*, with the corresponding CSV saved in the same folder.

In the plot, the vertical dashed line marks the takeover point. Before this point, the trust for the claimed user stays in a moderate band. After the switch, the claimed-user trust drops sharply and remains close to the LOCK region for most of the attacker segment. This is the intended behaviour for a continuous authentication system: when the active typist changes, the enrolled-user trust should fall.

One important detail is how trust is defined in this figure. I plot trust for the claimed user, not generic maximum model confidence. If only maximum confidence is plotted, the graph can be misleading because the model may still be confident, but confident about the wrong user. With claimed-user trust, the takeover is visible as a direct drop.

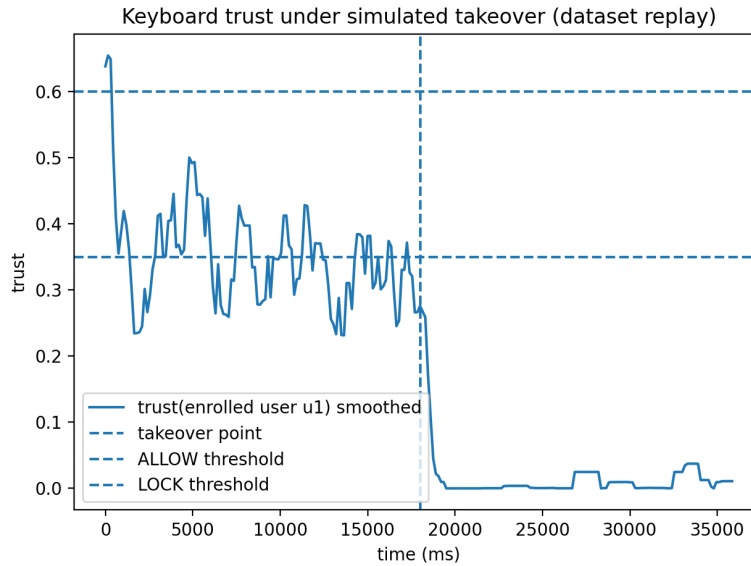


Figure 4.13 – Keyboard trust under simulated takeover using held-out test replay from *artifacts/realtime/takeover_sim_kb.png*

4.6 Real time prototype evidence

Along with offline metrics, I include evidence from the live Behavioural Trust Monitor because this project is about continuous scoring, not only static tables. The prototype runs in the browser, captures keyboard and mouse activity, sends window features to Django endpoints, and displays keyboard trust, mouse trust, behavioural trust, face trust and the final policy action.

The live time-series plot is generated from *artifacts/realtime/live_trust_timeseries.csv* and saved as *artifacts/realtime/fig_trust_timeseries.png*. It shows that the trust values change over time and that the fused score stays more stable than some individual signals. I also include the action-count bar chart from *artifacts/realtime/fig_actions_bar.png*, which shows how much time the session spent in ALLOW, STEP UP and LOCK.

I also carried out a small labelled live-stream test using separate genuine and takeover recordings. The test contained 822 genuine rows and 845 takeover rows. The live policy accuracy was 75.6%. The genuine allow rate was 95.5%, with 4.5% false alarms. The takeover detection rate was 56.2%, counted as STEP UP or LOCK. This shows the prototype was stable in this recorded genuine session, but takeover detection was still moderate.

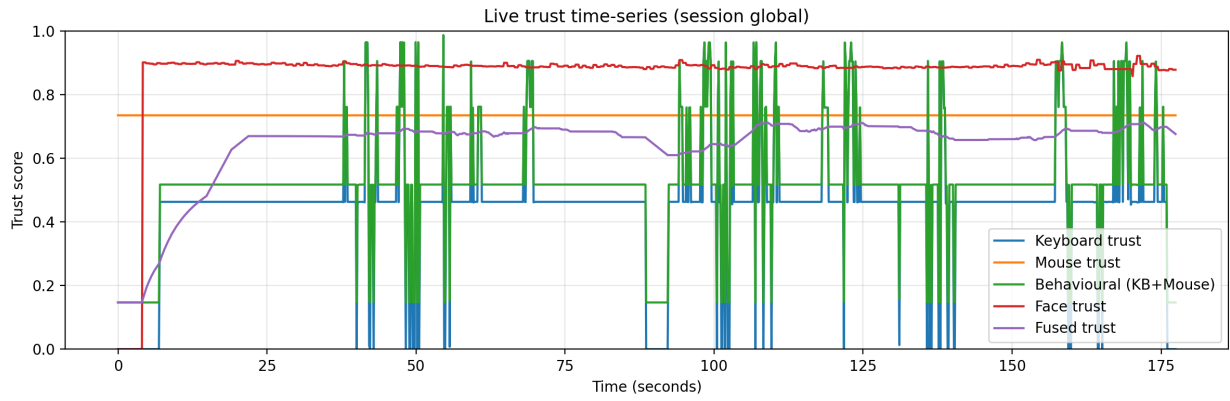


Figure 4.14 – Live trust time-series from artifacts/realtime/fig_trust_timeseries.png

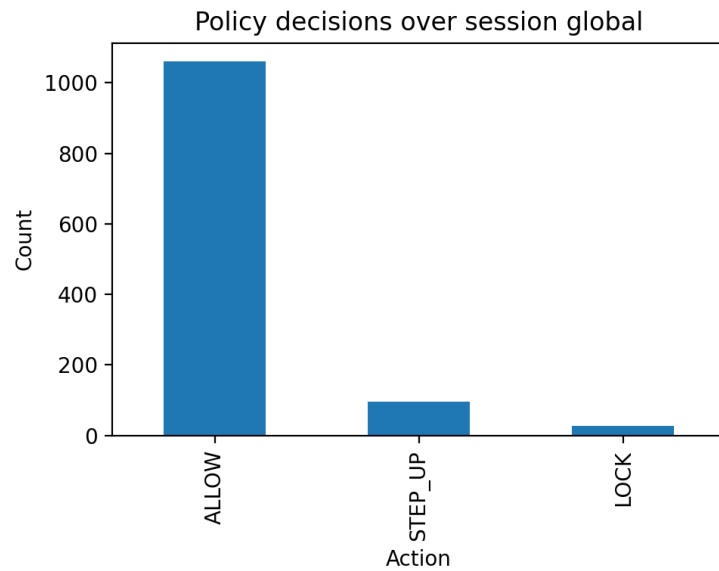


Figure 4.15 – Policy decision counts from artifacts/realtime/fig_actions_bar.png

CHAPTER 5 – DISCUSSION

5.1 Revisit objectives: what was met and what was not

If I check the objectives against the Chapter 4 evidence, the build objectives are mostly achieved. The system captures keyboard and mouse behaviour in short windows, scores them using saved models, fuses the available signals, and shows changing trust values in the live monitor. The logging also works, so the result is not only a visual demo. I can point to time-series plots, trust histograms, action counts, takeover replay and labelled live-stream metrics as evidence.

The part that is only partly achieved is reliable usability and takeover sensitivity together. The keyboard SVM gives very low FAR, which is good for rejecting impostors, but its FRR is high, so many genuine windows are also treated as weak. Mouse is not strong enough alone either, because the held-out result has high FRR and mouse activity can be missing during normal use. The labelled live test showed a 95.5% genuine allow rate but only 56.2% takeover detection. So the prototype was stable in the recorded genuine live session, but takeover detection was still moderate. This fits the wider security-usability trade-off discussed in continuous authentication research (Baig and Eskeland, 2021; Ryu et al., 2021).

5.2 Keyboard: why SVM was used instead of CAE in the final runtime

The CAE baseline was attractive because it can be trained per user and used like a verification model. In my implementation, each user has an autoencoder trained on genuine windows only. The model then uses reconstruction residual and a user-specific threshold to decide whether a new window looks genuine or suspicious. This approach gave useful separation in the corrected evaluation, with EER around 0.164 and ROC AUC around 0.908. However, it also introduced extra operational complexity because each user needs a separate model and threshold. Noisy behavioural windows can also affect reliability, which is why careful preprocessing and outlier handling are important in keystroke authentication work (Ismail et al., 2024).

The SVM approach was easier to control in this project. It uses one global scaler and one global model trained across users. That reduces the number of moving parts during live runtime scoring. Keystroke dynamics surveys support timing-based features for authentication, but the decision to use this SVM path is mainly based on my implementation results and integration needs (Teh et al., 2013; Shanmugapriya and Padmavathi, 2009). The trade-off is still visible. The keyboard SVM achieved very low FAR, which is useful for rejecting impostors, but its FRR was high, so it was strict with genuine windows. Because of this, I do not treat keyboard as a final single gate. I use it as the strongest behavioural stream inside fusion. A one-class setup could be explored later, and one-class SVM work shows that modelling the genuine user separately is a possible direction (Eude and Chang, 2018).

5.3 Mouse: why it is weaker alone and how it still helps in fusion

Mouse behaviour is weaker as a standalone signal in this project. The held-out mouse result had FAR around 0.084 but FRR around 0.750, which means many genuine mouse windows were rejected. This matches the practical issue with mouse dynamics: the signal depends strongly on whether the user is actively moving the pointer. If the user is reading, typing, or waiting for a page to load, there may be little useful mouse evidence.

This does not mean mouse is useless. It still helps as a support channel because it gives another behavioural view of the session. The mouse model uses movement features such as path length, speed, acceleration, jerk, direction changes and pause behaviour. Mouse dynamics work supports the idea that pointer behaviour can be used for authentication and anomaly detection, but the results also need to be interpreted carefully because the available interaction can vary by task and session (Khan et al., 2024; Almalki, Assery and Roy, 2021).

In the final prototype, I do not allow mouse to act as a standalone gatekeeper. Instead, mouse trust is fused with keyboard trust using a lower weight. This makes the system more cautious. If mouse evidence is present, it can support the behavioural trust score. If mouse evidence is missing or weak, the system should not immediately punish the user. This is why the backend uses short score holding and slow decay for missing mouse evidence.

5.4 Fusion and policy: value and limitations

Fusion and policy are needed because the single-channel results are not strong enough for a smooth continuous system. Keyboard is useful for rejecting impostors, but it is strict and rejects many genuine windows. Mouse is not strong enough alone either, because the held-out evaluation had high FRR and mouse evidence may be missing during normal use. Combining them is therefore a practical middle path. In my implementation, fusion is done using simple score-level rules with fixed weights. I did not use adaptive fusion because that would need more users, more sessions and stronger calibration evidence. I kept the logic transparent so the report can explain exactly what the system is doing.

The three-level policy also makes practical sense. If trust is clearly high, the session should stay in ALLOW. If trust is clearly low, LOCK is reasonable. The difficult part is the middle range, where the system is unsure. That is why STEP UP exists. It gives the prototype a safer middle action instead of jumping directly from allow to lock. This follows the general security-usability trade-off discussed in continuous authentication literature (Baig and Eskeland, 2021; Ryu et al., 2021).

The limitation is calibration. Small threshold changes can alter the number of STEP UP and LOCK events, especially when trust scores cluster near a boundary. Smoothing also changes behaviour. It reduces jitter, but it can delay takeover detection. The policy works as a prototype, but it would need proper calibration before real deployment.

5.5 Threats to validity and limitations

There are several limitations I must be open about. First is the impostor model. In offline evaluation, impostor windows are other users from the same dataset. This is a reasonable baseline, but it does not represent a skilled attacker who tries to mimic typing or uses automation. So the FAR values should be read as dataset-level impostor results, not as a full adversarial guarantee.

Second is dataset constraints. Both keyboard and mouse models are built from public datasets. They give a useful starting point, but they cannot represent every keyboard type, workstation setup, device, or work task. Behaviour drift is also a real issue. A user may type or move differently when tired, stressed, or using a different device, which can increase false rejection in practice. Keystroke dynamics literature also discusses variability and challenges in user behaviour, so this limitation is expected (Teh et al., 2013).

Third is the face liveness limitation. My liveness score is basic and motion-based. It was not evaluated against print, replay, mask, or AI-generated face attacks. Face anti-spoofing surveys make it clear that reliable liveness needs dedicated methods, datasets and benchmarking (Xing et al., 2025). So face is only a support signal in this project.

Finally, the live testing is small. The labelled live-stream test is useful because it uses unseen browser activity, but it is not a large user study. Trust scores are also not calibrated probabilities. They depend on model margins, scaling, thresholds and session conditions. For this reason, I treat the system as a proof-of-concept and recommend larger live testing, threshold calibration and user adaptation as future work.

CHAPTER 6 – EVALUATION, REFLECTIONS, AND CONCLUSIONS

6.1 Overall evaluation against the proposal plan

In my proposal, the target was a hybrid continuous authentication system where keyboard and mouse work silently in the background, and face-based evidence can support stronger checks when behaviour looks suspicious. If I evaluate the final submission against that plan, the core idea is delivered. The system runs as a web prototype. The browser captures typing and mouse movement, converts them into short windows, and sends them to the backend. The backend loads the saved models, generates per-modality trust values, fuses them, applies smoothing, and returns a policy decision as ALLOW, STEP UP, or LOCK. Most importantly, it is not only a demo. It generates logs and plots that I can use as evidence in the report.

A few things changed during implementation because the original plan was too ambitious in some places. The biggest change was the keyboard model. I initially considered a per-user autoencoder as the main keyboard verifier. After correction, the CAE baseline gave useful separation, with EER around 0.164 and ROC AUC around 0.908. However, it still needed one model and one threshold for each user, which made the runtime path more complicated. The SVM approach was easier to integrate into the live monitor because it uses one global model and scaler. Because of this, I kept the CAE as a baseline and used the keyboard SVM as the final runtime scorer.

The second change was about face. In the proposal, face was part of the multimodal design, but in the final implementation I treated it as step-up support rather than a fully trained face-authentication system. A proper face anti-spoofing setup needs dedicated datasets and evaluation that I could not complete properly in the project scope. So I kept the face pipeline lightweight, used a match score and a basic liveness score, and stated the limitation clearly instead of making strong claims.

Overall, the final design is slightly more conservative than the proposal, but stronger as evidence. Metrics CSVs, the takeover plot, live time-series plots, histograms, action counts and labelled live-stream metrics show what the system actually does, not only what it was supposed to do.

6.2 Reflection: what I learned and what I would do differently

This project taught me that continuous authentication is not only about choosing a strong model. It is mainly about making the full pipeline behave properly in real time. Training a model and getting an offline metric is only one part of the work. Making the model handle missing input, noisy short windows, idle periods and real user behaviour is where most of the practical difficulty appeared.

One important learning was about trust scoring. At first, I used simple maximum-probability style confidence. It looked acceptable in offline checks, but in the live monitor the trust sometimes appeared constant or misleading. After debugging and testing, I understood that

continuous authentication needs a trust definition that matches the actual security question. The important question is whether the current operator still looks like the enrolled user. It is not only whether the model is confident about any class. This is why, for keyboard, I used margin-based trust, and for the takeover plot I used claimed-user trust rather than generic maximum confidence.

Another learning was that small engineering choices can change the full system behaviour. Window timing, feature order, scaling, and how the browser handles idle windows can break the system silently. I faced feature mismatch issues and trust values not updating properly. I had to fix these by keeping feature dimensions consistent, reusing the correct saved scaler, and avoiding fake zero scores during idle periods. This part is not glamorous, but it is the core reason why the prototype works.

If I had more time, I would improve three things. First, I would calibrate thresholds based on real costs, not only selected operating points. Second, I would add adaptation, because behaviour can drift over time and fixed models may increase FRR in real use. Third, I would improve face evaluation. I would either keep face only as a minor support signal or test it properly with standard anti-spoofing datasets, because the current liveness score is not strong enough to claim robust spoof resistance.

6.3 Conclusion: key takeaways and future work

The main takeaway from this dissertation is that continuous authentication can act as a practical safety net for session hijacking, but only if it is treated as a risk-scoring system, not as a perfect biometric lock. The keyboard SVM results show a clear trade-off. FAR is very low, so it is useful for rejecting impostors, but FRR is high, so genuine windows can also be rejected often. This means keyboard alone is not suitable for harsh locking behaviour. Mouse adds useful behavioural support, but the held-out result also shows high FRR, and in real sessions mouse activity is not always available. So mouse alone is not enough either.

The part that works best in a realistic sense is combining signals with a clear fusion rule and using a three-level policy. STEP UP is important because it gives the system a middle option instead of jumping straight from ALLOW to LOCK. The takeover replay also supports the project goal because it shows that when claimed-user trust drops after a user switch, the system can move toward a stronger response.

For future work, I would focus on making the system more deployable. This means better threshold calibration, improved handling of idle periods, and context-aware weighting instead of fixed weights. I would also strengthen face by using proper liveness methods and evaluation. Finally, I would run a small user study with multiple sessions per user and different devices. Without that, the system remains a prototype with evidence, but it cannot claim strong generalisation to real workplace conditions.

REFERENCES

- Wang, X. and Hou, D. (2024) 'Enhancing Keystroke Dynamics Authentication with Ensemble Learning and Data Resampling Techniques', *Electronics.*, 13(22). Available at: <https://doi.org/10.3390/electronics13224559>.
- Killourhy, K.S. and Maxion, R.A. (2009) 'Comparing anomaly-detection algorithms for keystroke dynamics', 2009 IEEE/IFIP International Conference on Dependable Systems and Networks, Lisbon, Portugal, pp. 125–134. Available at: <https://doi.org/10.1109/DSN.2009.5270346>.
- Khan, S. *et al.* (2024) 'Mouse Dynamics Behavioral Biometrics: A Survey', *ACM computing surveys*, 56(6), pp. 1–33. Available at: <https://doi.org/10.1145/3640311>.
- AbdelRaouf, H., Chelloug, S.A., Muthanna, A., Semary, N., Amin, K. and Ibrahim, M. (2023) 'Efficient Convolutional Neural Network-Based Keystroke Dynamics for Boosting User Authentication', *Sensors*, 23(10), 4898. Available at: <https://doi.org/10.3390/s23104898>.
- Almalki, S., Assery, N. and Roy, K. (2021) 'An Empirical Evaluation of Online Continuous Authentication and Anomaly Detection Using Mouse Clickstream Data Analysis', *Applied sciences*, 11(13). Available at: <https://doi.org/10.3390/app11136083>.
- Antal, M. and Egyed-Zsigmond, E. (2019) 'Intrusion detection using mouse dynamics', *IET biometrics*, 8(5), pp. 285–294. Available at: <https://doi.org/10.1049/iet-bmt.2018.5126>.
- Teh, P.S. *et al.* (2013) 'A Survey of Keystroke Dynamics Biometrics', *TheScientificWorld.*, 2013(1). Available at: <https://doi.org/10.1155/2013/408280>.
- Shanmugapriya, D. and Padmavathi, G., 2009. A survey of biometric keystroke dynamics: Approaches, security and challenges. *arXiv preprint arXiv:0910.0817*. Available at: <https://doi.org/10.48550/arXiv.0910.0817>.
- Shrivastava, A. and Bollavarapu, S. (2025) *Exploring Keystroke Dynamics: Enhancing Authentication Through Typing Patterns*. Atlantis Press International BV, pp. 3127–3140. Available at: https://doi.org/10.2991/978-94-6463-858-5_261.
- Xiaofeng, L., Shengfei, Z. and Shengwei, Y. (2019) 'Continuous authentication by free-text keystroke based on CNN plus RNN', *Procedia computer science.*, 147, pp. 314–318.

Available at: <https://doi.org/10.1016/j.procs.2019.01.270>.

- Li, J., Chang, H.-C. and Stamp, M. (2022) *Free-Text Keystroke Dynamics for User Authentication*. Springer International Publishing, pp. 357–380.
Available at: https://doi.org/10.1007/978-3-030-97087-1_15.
- IEEE Conference on Biometrics: Theory, A. (2016) ‘User authentication with keystroke dynamics in long-text data’, in *BTAS : 2016 IEEE 8th International Conference on Biometrics Theory, Applications and Systems : 6-9 September 2016*. New York : IEEE, pp. 1396–6.
Available at: <https://doi.org/10.1109/BTAS.2016.7791182>.
- Alsultan, A., Warwick, K. and Wei, H. (2017) ‘Non-conventional keystroke dynamics for user authentication’, *Pattern recognition letters.*, 89, pp. 53–59.
Available at: <https://doi.org/10.1016/j.patrec.2017.02.010>.
- ICB Hong Kong, C.(: and ICB (Conference) Corporate Author (2005) *GA SVM Wrapper Ensemble for Keystroke Dynamics Authentication, Advances in biometrics : International Conference, ICB 2006, Hong Kong, China, January 5-7, 2006 : proceedings* /. Berlin : Springer, pp. 654–660.
Available at: https://doi.org/10.1007/11608288_87.
- G Ismail, M. *et al.* (2024) ‘Outlier detection for keystroke biometric user authentication’, *PeerJ.*, 10.
Available at: <https://doi.org/10.7717/peerj-cs.2086>.
- Eude, T. and Chang, C. (2018) ‘One-class SVM for biometric authentication by keystroke dynamics for remote evaluation’, *Computational intelligence.*, 34(1), pp. 145–160.
Available at: <https://doi.org/10.1111/coin.12122>.
- Ryu, R. *et al.* (2021) ‘Continuous Multimodal Biometric Authentication Schemes: A Systematic Review’, *IEEE access : practical innovations, open solutions.*, 9, pp. 34541–34557.
Available at: <https://doi.org/10.1109/ACCESS.2021.3061589>.
- Baig, A.F. and Eskeland, S. (2021) ‘Security, Privacy and Usability in Continuous Authentication: A Survey’, *Sensors*, 21(17).
Available at <https://doi.org/10.3390/s21175967>.
- Stylios, I.C. *et al.* (2016) ‘A Review of Continuous Authentication Using Behavioral Biometrics’, in *Proceedings of the SouthEast European Design Automation, Computer Engineering, Computer Networks and Social Media Conference*. ACM, pp. 45–79.
Available at: <https://doi.org/10.1145/2984393.2984403>.

- Wang, Y., Wu, C., Liao, Y. and You, M., 2025. Optimizing Mouse Dynamics for User Authentication by Machine Learning: Addressing Data Sufficiency, Accuracy-Practicality Trade-off and Model Performance Challenges. *arXiv preprint arXiv:2504.21415*.
Available at: <https://doi.org/10.48550/arXiv.2504.21415>.
- Almalki, S., Assery, N. and Roy, K. (2021) 'An Empirical Evaluation of Online Continuous Authentication and Anomaly Detection Using Mouse Clickstream Data Analysis', *Applied sciences*, 11(13).
Available at: <https://doi.org/10.3390/app11136083>.
- Ketab, S.S., Clarke, N.L. and Dowland, P.S., 2017. A robust e-invigilation system employing multimodal biometric authentication. *International Journal of Information and Education Technology*, 7(11), p.796.
Available at: <https://doi.org/10.18178/ijiet.2017.7.11.975>
- S, A. and K, J.S. (2025) 'Enhancing security and usability with context aware multi-biometric fusion for continuous user authentication', *Scientific reports.*, 15(1).
Available at: <https://doi.org/10.1038/s41598-025-14833-z>.
- Martín, A.G. *et al.* (2022) 'Combining user behavioural information at the feature level to enhance continuous authentication systems', *Knowledge-based systems.*, 244.
Available at: <https://doi.org/10.1016/j.knosys.2022.108544>.
- Zhang, K. *et al.* (2016) 'Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks', *IEEE signal processing letters.*, 23(10), pp. 1499–1503. Available at: <https://doi.org/10.1109/LSP.2016.2603342>.
- Manoj G *et al.* (2022) 'A Survey on Anti-Spoofing Methods for Facial Recognition', *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp. 259–268.
Available at: <https://doi.org/10.32628/CSEIT228248>.
- Xing, H. *et al.* (2025) 'Face Anti-Spoofing Based on Deep Learning: A Comprehensive Survey', *Applied sciences*, 15(12).
Available at: <https://doi.org/10.3390/app15126891>.
- Garabato, D. *et al.* (2022) 'AI-based user authentication reinforcement by continuous extraction of behavioral interaction features', *Neural computing & applications.*, 34(14), pp. 11691–11705. Available at: <https://doi.org/10.1007/s00521-022-07061-3>.

APPENDIX A – SYSTEM DIAGRAMS AND UI SCREENSHOTS

A.1 System Architecture and Workflow

Overall block diagram of browser inputs, backend components, models, fusion, and decisions.

Behavioral Trust Monitor

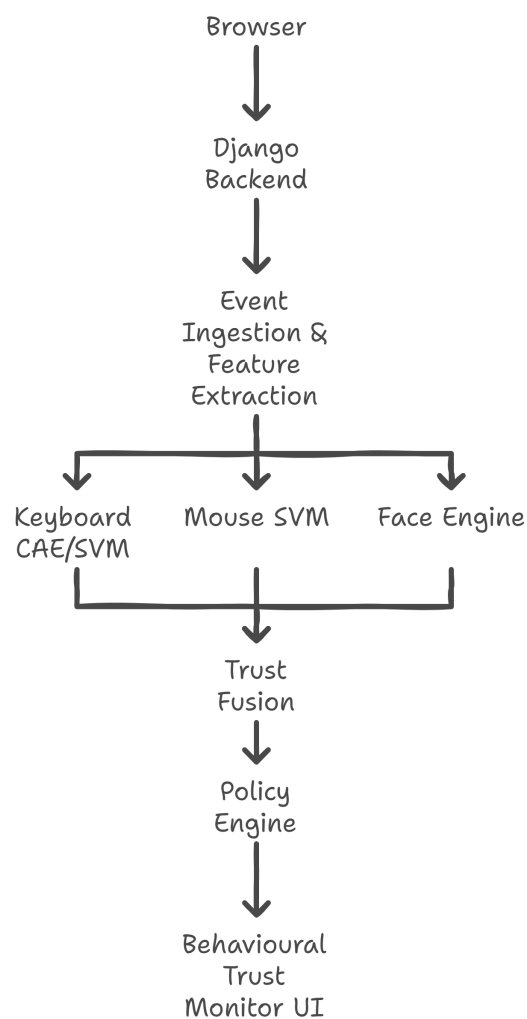


Figure A.1 – High-level System Architecture

End-to-end dataflow from raw events through preprocessing, training, online scoring and UI.

Behavioural Trust Monitor System Flow

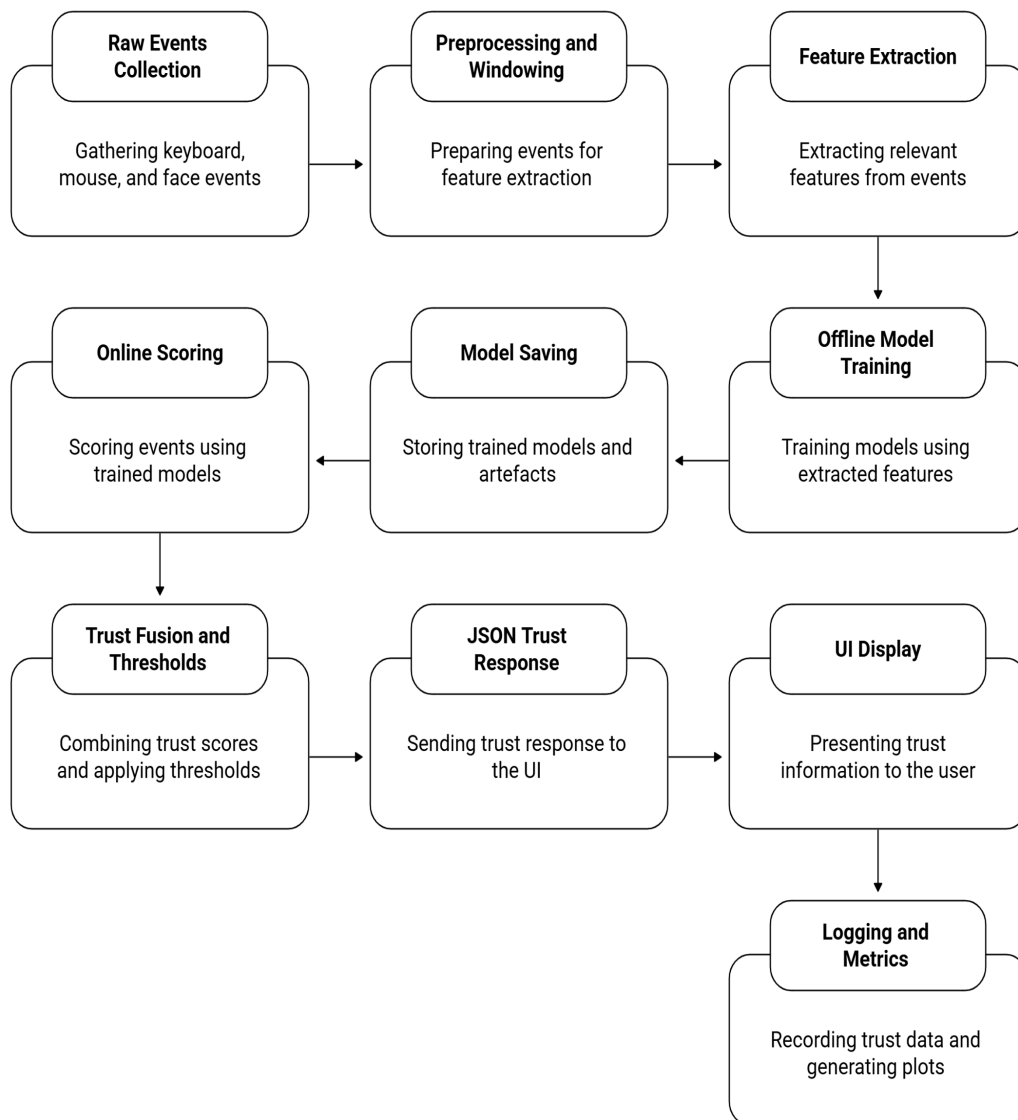


Figure A.2 – End-to-End System Workflow

A.2 Behavioural Trust Monitor UI

live monitoring page showing per-channel trust values, time-series traces and current policy state.

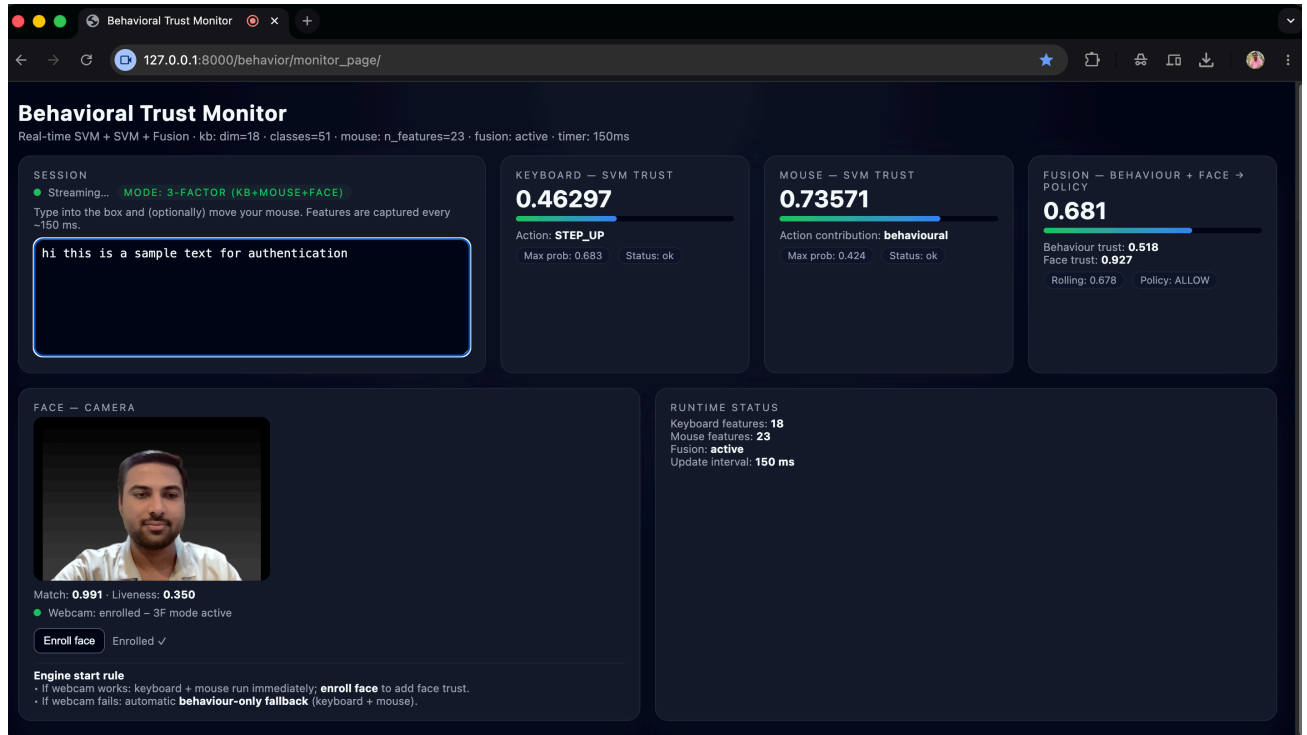


Figure A.3 – Behavioural Trust Monitor user interface

APPENDIX B DEFINITIONS

B.1 Keyboard feature vector (18 features, fixed order)

Source of feature order: matches Data/keyboard_train_windows.csv, Data/keyboard_val_windows.csv, and Data/keyboard_test_windows.csv.

Non-feature columns in the CMU window file: user_id, session_id, window_id.

Units note: In the prepared keyboard split files, timing features come from the CMU-style dataset columns and are treated as numeric timings. In the live browser, timings are built from timestamps and then converted into the same 18D layout.

Feature list and definition:

1. **dwel_mean:** mean of key hold times (keydown to keyup)
2. **dwel_std:** standard deviation of hold times
3. **dwel_p10:** 10th percentile of hold times
4. **dwel_p50:** median of hold times
5. **dwel_p90:** 90th percentile of hold times
6. **dd_mean:** mean of down-down gaps (time between consecutive keydown events)
7. **dd_std:** standard deviation of down-down gaps
8. **dd_p10:** 10th percentile of down-down gaps
9. **dd_p50:** median of down-down gaps
10. **dd_p90:** 90th percentile of down-down gaps
11. **ud_mean:** mean of up-down gaps (keyup to next keydown)
12. **ud_std:** standard deviation of up-down gaps
13. **ud_p10:** 10th percentile of up-down gaps
14. **ud_p50:** median of up-down gaps
15. **ud_p90:** 90th percentile of up-down gaps
16. **backspace_rate:** fraction of keys that are Backspace within the window
17. **burst_mean:** average burst run-length based on short DD gaps (DD < 0.150s treated as within-burst)
18. **idle_frac:** fraction of large gaps based on UD (UD > 1.0s treated as idle-like)

B.2 Mouse feature vector (23 features, fixed order)

Source of feature order: /Models/mouse/mouse_meta.json under key features.

Raw input: mouse session points with columns (time ms, x, y).

Core computation: based on Apps/behavior/mouse_ingest.py kinematics.

1. **dur_ms**: duration of the window in milliseconds ($t_{\text{last}} - t_{\text{first}}$)
2. **n_points**: number of points in the window
3. **path_len**: sum of Euclidean distances between consecutive points
4. **straight_len**: straight-line distance from first point to last point
5. **straightness**: $\text{straight_len} / \text{path_len}$ (0 if path_len is ~ 0)
6. **mean_speed**: mean speed over segments (distance / dt)
7. **p95_speed**: 95th percentile of speed
8. **max_speed**: maximum speed
9. **mean_acc**: mean acceleration (delta speed / dt)
10. **p95_acc**: 95th percentile of acceleration
11. **max_acc**: maximum acceleration
12. **mean_jerk**: mean jerk (delta acceleration / dt)
13. **p95_jerk**: 95th percentile of jerk
14. **max_jerk**: maximum jerk
15. **dx**: net x displacement ($x_{\text{last}} - x_{\text{first}}$)
16. **dy**: net y displacement ($y_{\text{last}} - y_{\text{first}}$)
17. **abs_dx**: absolute net x displacement
18. **abs_dy**: absolute net y displacement
19. **bbox_w**: bounding box width ($\max(x) - \min(x)$)
20. **bbox_h**: bounding box height ($\max(y) - \min(y)$)
21. **bbox_area**: $\text{bbox_w} * \text{bbox_h}$
22. **direction_changes**: count of sign changes in dx and dy across segments (more zig-zag gives higher)
23. **pause_ratio_20ms**: fraction of dt values greater than 20 ms

B.2 Keyboard trust conversion using logistic squashing

The keyboard SVM produces decision scores rather than a direct trust value. In the live runtime scorer, I use the gap between the top predicted class score and the second highest score as a confidence margin. A larger margin means the model is more clearly separating the current keyboard pattern from other users. This margin is passed through a logistic squashing function to convert it into a trust value between 0 and 1. This keeps the live trust display bounded and smoother than using raw SVM scores. If the margin is missing, invalid or not positive, the trust is treated as low.

APPENDIX C MODEL CONFIGS, HYPERPARAMETERS, AND TRAINING COMMANDS

C.1 Keyboard CAE baseline

Purpose: The keyboard CAE is used as a per-user reconstruction baseline for keyboard verification. It is not the final live runtime model.

Code files:

- Apps/behavior/train_kb_cae_cmu.py
- Apps/behavior/eval_kb_cae.py
- Apps/behavior/Plots/build_kb_cae_scores.py
- Apps/behavior/Plots/plot_kb_cae_histograms.py
- Apps/behavior/Plots/plot_kb_cae_far_frr.py
- Apps/behavior/Plots/plot_kb_cae_roc.py
- Apps/behavior/Plots/plot_kb_cae_pr.py
- Apps/behavior/Plots/plot_kb_cae_confusion_eer.py

Data used:

- Data/keyboard_train_windows.csv
- Data/keyboard_val_windows.csv
- Data/keyboard_test_windows.csv

Sample sizes:

- Users: 51
- Training windows: 12,240
- Validation windows: 4,080
- Held-out test windows: 4,080

Feature input:

- 18 keyboard features in fixed order

Model configuration:

- One autoencoder per user
- StandardScaler fitted only on that user's genuine training windows
- Input dimension: 18
- Encoder: Dense 64 → Dense 32 → Bottleneck Dense 16
- Decoder: Dense 32 → Dense 64 → Output Dense 18
- Loss: mean squared reconstruction error
- Optimiser: Adam
- Batch size: 256
- Epochs: up to 40
- Early stopping used where available

Thresholding and testing:

- Validation split used to choose reconstruction-error threshold tau
- Held-out test split used only for final reporting
- Reported metrics include FAR, FRR, ACC, EER, ROC AUC and PR area

Main outputs:

- Models/kb_cae/<user_id>/
- artifacts/kb_cae_metrics_explicit.csv
- artifacts/kb_cae_overview_explicit.json
- artifacts/kb_cae_scores_explicit.csv
- artifacts/plots/kb_cae_scores.csv
- artifacts/plots/kb_cae_trust_histograms.png
- artifacts/plots/kb_cae_far_frr_vs_threshold.png
- artifacts/plots/kb_cae_roc_curve.png
- artifacts/plots/kb_cae_pr_curve.png
- artifacts/plots/kb_cae_confusion_eer.png

Commands run:

- python Apps/behavior/train_kb_cae_cmu.py
- python Apps/behavior/eval_kb_cae.py
- python Apps/behavior/Plots/build_kb_cae_scores.py
- python -m Apps.behavior.Plots.plot_kb_cae_histograms
- python -m Apps.behavior.Plots.plot_kb_cae_far_frr
- python -m Apps.behavior.Plots.plot_kb_cae_roc
- python -m Apps.behavior.Plots.plot_kb_cae_pr
- python -m Apps.behavior.Plots.plot_kb_cae_confusion_eer

C.2 Keyboard SVM final runtime model

Purpose: The keyboard SVM is the final keyboard model used in the live Behavioural Trust Monitor.

Code files:

- Apps/behavior/eval_kb_svm.py
- Apps/behavior/ae_conditional.py

- Apps/behavior/Plots/keyboard/eval_keyboard_scores.py
- Apps/behavior/Plots/keyboard/eval_keyboard_classification.py

Data used:

- Data/keyboard_train_windows.csv
- Data/keyboard_val_windows.csv
- Data/keyboard_test_windows.csv

Sample sizes:

- Users: 51
- Training rows: 12,240
- Validation rows: 4,080
- Held-out test rows: 4,080
- Final training after hyperparameter choice: 16,320 rows from train + validation

Feature input:

- 18 keyboard features in fixed order

Model configuration:

- Model: sklearn.svm.SVC
- Kernel: RBF
- Class balancing: class_weight="balanced"
- Global StandardScaler fitted on training windows only
- Scaler reused for validation, test and live runtime scoring

Hyperparameter tuning:

The hyperparameters were chosen using the validation split, not the held-out test split. I tested different C and gamma values for the RBF SVM and selected the pair that gave the best validation behaviour. The final selected values were $C = 20.0$ and $\gamma = 0.1$. After selecting them, the final model was trained using the combined training and validation rows, while the test split was kept only for final evaluation. This avoids tuning directly on the test data.

- Validation grid over C and gamma
- Final selected values:
- $C = 20.0$
- $\gamma = 0.1$

Held-out verification setup:

- 80 genuine test windows per user
- 4,000 impostor test windows per user
- Reported metrics: FAR, FRR and ACC

Main outputs:

- Models/kb_svm/
- artifacts/kb_svm_eval_metrics.csv
- artifacts/kb_svm_eval_overview.json
- artifacts/keyboard/kb_scores.csv
- artifacts/keyboard/kb_svm_metrics.csv

- artifacts/keyboard/kb_svm_confusion.png
- artifacts/keyboard/kb_svm_pr.png

Commands run:

- python Apps/behavior/eval_kb_svm.py
- python -m Apps.behavior.Plots.keyboard.eval_keyboard_scores
- python -m Apps.behavior.Plots.keyboard.eval_keyboard_classification

C.3 Mouse SVM model

Purpose: The mouse SVM is used as a supporting behavioural signal in fusion. It is not used as a standalone lock decision source

Code files:

- Apps/behavior/train_mouse_svm.py
- Apps/behavior/eval_mouse.py
- Apps/behavior/Plots/mouse/eval_mouse_scores.py

Data used:

- Data/mouse_windows_train.csv
- Data/mouse_windows_train_augmented.csv
- Data/mouse_windows_test.csv

Sample sizes:

- Tuning file: 23,358 windows
- Users: 10
- Raw files used for grouping: 65
- Held-out test file: 23,792 windows

Feature input:

- 23 mouse features in fixed order

Model configuration:

- Model: sklearn.svm.SVC
- Kernel: RBF
- Class balancing: class_weight="balanced"
- Probability outputs enabled: probability=True
- Global StandardScaler fitted on training windows
- Feature order saved in mouse_meta.json

Hyperparameter tuning:

For the mouse model, hyperparameters were selected using grouped cross-validation by raw file. This was important because windows from the same raw mouse recording can be very similar. Grouping by raw file reduced the risk of choosing parameters that only worked because near-duplicate windows

appeared in both training and validation folds. The final selected values were $C = 50.0$ and $\gamma = 0.1$.

- Grouped cross-validation by raw file
- Final selected values:
- $C = 50.0$
- $\gamma = 0.1$
- Grouped CV mean accuracy: 0.3256
- Grouped CV mean macro F1: 0.2670

Important note:

In the final run, `mouse_windows_train_augmented.csv` had the same row count as the base training file, so I do not claim that augmentation increased the dataset size.

Main outputs:

- `Models/mouse/mouse_svm.joblib`
- `Models/mouse/mouse_scaler.joblib`
- `Models/mouse/mouse_meta.json`
- `artifacts/mouse_svm_eval_metrics.csv`
- `artifacts/mouse_svm_eval_overview.json`
- `artifacts/mouse/mouse_scores.csv`
- `artifacts/mouse/mouse_roc.png`
- `artifacts/mouse/mouse_far_frr.png`
- `artifacts/mouse/mouse_hist.png`
- `artifacts/mouse/mouse_svm_confusion.png`
- `artifacts/mouse/mouse_svm_pr.png`

Commands run:

- `python Apps/behavior/train_mouse_svm.py`
- `python Apps/behavior/eval_mouse.py`
- `python Apps/behavior/Plots/mouse/eval_mouse_scores.py`

C.4 Runtime fusion and policy

Purpose: Runtime fusion combines keyboard, mouse and optional face scores into the final trust score and policy action.

Code files:

- `Apps/behavior/trust_fusion.py`
- `Apps/behavior/views.py`

Final fusion settings:

- $\text{behaviour_trust} = 0.8 * \text{keyboard_trust} + 0.2 * \text{mouse_trust}$
- $\text{behaviour_trust} = \max(\text{behaviour_trust}, \text{keyboard_trust})$
- $\text{face_trust} = 0.9 * \text{face_match} + 0.1 * \text{face_liveness}$
- $\text{raw_overall_trust} = 0.6 * \text{behaviour_trust} + 0.4 * \text{face_trust}$
- $\text{overall_trust} = \max(\text{behaviour_trust}, \text{raw_overall_trust})$

Policy thresholds:

- $\text{overall_trust} < 0.30$ -> LOCK
- $0.30 \leq \text{overall_trust} < 0.55$ -> STEP_UP
- $\text{overall_trust} \geq 0.55$ -> ALLOW

Important note:

These weights and thresholds are implementation choices from this prototype. They are not copied from the literature.

C.5 Takeover replay generation

Purpose: The takeover replay shows how claimed-user keyboard trust changes when the active typist switches.

Code file:

- `make_takeover_plot.py`

Input:

- Held-out keyboard test windows
- Saved keyboard SVM model and scaler

Main outputs:

- `artifacts/realtime/takeover_sim_kb.csv`
- `artifacts/realtime/takeover_sim_kb.png`

Command run:

- `python make_takeover_plot.py`

C.6 Real-time session plots

Purpose: This script generates runtime trust plots from live browser logs.

Code file:

- `Apps/behavior/eval_realtime_plots.py`

Input:

- `artifacts/realtime/live_trust_timeseries.csv`

Main outputs:

- `artifacts/realtime/fig_trust_timeseries.png`
- `artifacts/realtime/fig_trust_histograms.png`
- `artifacts/realtime/fig_actions_bar.png`
- `artifacts/realtime/trust_summary.csv`

Command run:

- `python Apps/behavior/eval_realtime_plots.py`

C.7 Labelled live-stream policy test

Purpose: The labelled live test checks policy behaviour on unseen browser activity using separate genuine and takeover recordings.

Code file:

- Apps/behavior/eval_live_labelled.py

Input files:

- artifacts/realtime/live_genuine.csv
- artifacts/realtime/live_takeover.csv

Main output:

- artifacts/realtime/live_labelled_policy_metrics.csv

Final labelled live result:

- Genuine rows: 822
- Takeover rows: 845
- Live policy accuracy: 75.6%
- Genuine allow rate: 95.5%
- Genuine false alarm rate: 4.5%
- Takeover detection rate: 56.2%

Commands used during recording and evaluation:

- rm artifacts/realtime/live_trust_timeseries.csv
- python manage.py runserver
- cp artifacts/realtime/live_trust_timeseries.csv artifacts/realtime/live_genuine.csv
- rm artifacts/realtime/live_trust_timeseries.csv
- python manage.py runserver
- cp artifacts/realtime/live_trust_timeseries.csv artifacts/realtime/live_takeover.csv
- python Apps/behavior/eval_live_labelled.py

APPENDIX D –EXECUTION EVIDENCE SCREENSHOTS

This appendix provides terminal and runtime screenshots showing that the main training, evaluation, plotting and live-policy scripts were executed. The screenshots are included as supporting evidence for the methods and results reported in Chapters 3 and 4.

D.1 Keyboard CAE training and evaluation

Figure D.1 – Keyboard CAE training output showing 51 users retained, per-user CAE training, and saved training summary files.

```
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %  
python Apps/behavior/train_kb_cae_cmu.py  
[INFO] Keyboard CAE users retained: 51  
[INFO] Trained CAE for s002: n_train=240, n_val=80, epochs=40, best_val_loss=0.574974  
[INFO] Trained CAE for s003: n_train=240, n_val=80, epochs=40, best_val_loss=0.544600  
[INFO] Trained CAE for s004: n_train=240, n_val=80, epochs=40, best_val_loss=0.552169  
[INFO] Trained CAE for s005: n_train=240, n_val=80, epochs=40, best_val_loss=0.535409  
[INFO] Trained CAE for s007: n_train=240, n_val=80, epochs=40, best_val_loss=0.558111  
[INFO] Trained CAE for s008: n_train=240, n_val=80, epochs=40, best_val_loss=0.844455  
[INFO] Trained CAE for s010: n_train=240, n_val=80, epochs=40, best_val_loss=0.715769  
[INFO] Trained CAE for s011: n_train=240, n_val=80, epochs=40, best_val_loss=0.722812  
[INFO] Trained CAE for s012: n_train=240, n_val=80, epochs=40, best_val_loss=0.705969  
[INFO] Trained CAE for s013: n_train=240, n_val=80, epochs=40, best_val_loss=0.656260  
[INFO] Trained CAE for s015: n_train=240, n_val=80, epochs=40, best_val_loss=0.737001  
[INFO] Trained CAE for s016: n_train=240, n_val=80, epochs=40, best_val_loss=0.708553  
[INFO] Trained CAE for s017: n_train=240, n_val=80, epochs=40, best_val_loss=0.684839  
[INFO] Trained CAE for s018: n_train=240, n_val=80, epochs=40, best_val_loss=0.597925  
[INFO] Trained CAE for s019: n_train=240, n_val=80, epochs=40, best_val_loss=0.777065  
[INFO] Trained CAE for s020: n_train=240, n_val=80, epochs=40, best_val_loss=0.798310  
[INFO] Trained CAE for s021: n_train=240, n_val=80, epochs=40, best_val_loss=0.616635  
[INFO] Trained CAE for s022: n_train=240, n_val=80, epochs=40, best_val_loss=0.609405  
[INFO] Trained CAE for s024: n_train=240, n_val=80, epochs=40, best_val_loss=0.757971  
[INFO] Trained CAE for s025: n_train=240, n_val=80, epochs=40, best_val_loss=0.670193  
[INFO] Trained CAE for s026: n_train=240, n_val=80, epochs=40, best_val_loss=0.753378  
[INFO] Trained CAE for s027: n_train=240, n_val=80, epochs=40, best_val_loss=0.680195  
[INFO] Trained CAE for s028: n_train=240, n_val=80, epochs=40, best_val_loss=0.760389  
[INFO] Trained CAE for s029: n_train=240, n_val=80, epochs=40, best_val_loss=0.546731  
[INFO] Trained CAE for s030: n_train=240, n_val=80, epochs=40, best_val_loss=0.549459  
[INFO] Trained CAE for s031: n_train=240, n_val=80, epochs=40, best_val_loss=0.581317  
[INFO] Trained CAE for s032: n_train=240, n_val=80, epochs=40, best_val_loss=0.871010  
[INFO] Trained CAE for s033: n_train=240, n_val=80, epochs=40, best_val_loss=0.692400  
[INFO] Trained CAE for s034: n_train=240, n_val=80, epochs=40, best_val_loss=0.688231  
[INFO] Trained CAE for s035: n_train=240, n_val=80, epochs=40, best_val_loss=0.691324  
[INFO] Trained CAE for s036: n_train=240, n_val=80, epochs=40, best_val_loss=0.553129  
[INFO] Trained CAE for s037: n_train=240, n_val=80, epochs=40, best_val_loss=0.606002  
[INFO] Trained CAE for s038: n_train=240, n_val=80, epochs=40, best_val_loss=0.661065  
[INFO] Trained CAE for s039: n_train=240, n_val=80, epochs=40, best_val_loss=0.771723  
[INFO] Trained CAE for s040: n_train=240, n_val=80, epochs=40, best_val_loss=0.644878  
[INFO] Trained CAE for s041: n_train=240, n_val=80, epochs=40, best_val_loss=0.738779  
[INFO] Trained CAE for s042: n_train=240, n_val=80, epochs=40, best_val_loss=0.663353  
[INFO] Trained CAE for s043: n_train=240, n_val=80, epochs=40, best_val_loss=2.030591  
[INFO] Trained CAE for s044: n_train=240, n_val=80, epochs=40, best_val_loss=0.503216  
[INFO] Trained CAE for s046: n_train=240, n_val=80, epochs=40, best_val_loss=0.823031  
[INFO] Trained CAE for s047: n_train=240, n_val=80, epochs=40, best_val_loss=0.511017  
[INFO] Trained CAE for s048: n_train=240, n_val=80, epochs=40, best_val_loss=0.600275  
[INFO] Trained CAE for s049: n_train=240, n_val=80, epochs=40, best_val_loss=0.569781  
[INFO] Trained CAE for s050: n_train=240, n_val=80, epochs=40, best_val_loss=0.602592  
[INFO] Trained CAE for s051: n_train=240, n_val=80, epochs=40, best_val_loss=0.794155  
[INFO] Trained CAE for s052: n_train=240, n_val=80, epochs=40, best_val_loss=0.650644  
[INFO] Trained CAE for s053: n_train=240, n_val=80, epochs=40, best_val_loss=0.709296  
[INFO] Trained CAE for s054: n_train=240, n_val=80, epochs=40, best_val_loss=0.696571  
[INFO] Trained CAE for s055: n_train=240, n_val=80, epochs=40, best_val_loss=0.602315  
[INFO] Trained CAE for s056: n_train=240, n_val=80, epochs=40, best_val_loss=0.590181  
[INFO] Trained CAE for s057: n_train=240, n_val=80, epochs=40, best_val_loss=0.717507  
[INFO] Saved keyboard CAE summary -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments  
/artifacts/keyboard/kb_cae_train_summary.csv  
[INFO] Saved keyboard CAE summary -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments  
/artifacts/keyboard/kb_cae_train_summary.json  
[INFO] Keyboard CAE baseline training complete.  
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
```


Figure D.2 – Corrected keyboard CAE evaluation output showing per-user thresholds and saved kb_cae_metrics_explicit.csv and kb_cae_overview_explicit.json.

```
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-
[INFO] CAE baseline users retained: 51

[INFO] Training CAE for s002
tau=0.028565 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s003
tau=0.053648 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s004
tau=0.079143 FAR=0.001 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s005
tau=0.052545 FAR=0.000 FRR=0.988 ACC=0.981 ERR=0.494

[INFO] Training CAE for s007
tau=0.035879 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s008
tau=0.060825 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s010
tau=0.084001 FAR=0.000 FRR=0.975 ACC=0.981 ERR=0.488

[INFO] Training CAE for s011
tau=0.047531 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s012
tau=0.039702 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s013
tau=0.065176 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s015
tau=0.070068 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s016
tau=0.050435 FAR=0.000 FRR=0.975 ACC=0.981 ERR=0.487

[INFO] Training CAE for s017
tau=0.091953 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s018
tau=0.061078 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s019
tau=0.060798 FAR=0.000 FRR=0.938 ACC=0.982 ERR=0.469

[INFO] Training CAE for s020
tau=0.033656 FAR=0.001 FRR=0.988 ACC=0.980 ERR=0.494

[INFO] Training CAE for s021
tau=0.024489 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s022
tau=0.464598 FAR=0.003 FRR=0.425 ACC=0.989 ERR=0.214

[INFO] Training CAE for s024
tau=0.097990 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s025
tau=0.051836 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s026
tau=0.056235 FAR=0.001 FRR=0.975 ACC=0.980 ERR=0.488

[INFO] Training CAE for s027
tau=0.089158 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s031
tau=0.047486 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s032
tau=0.037408 FAR=0.001 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s033
tau=0.461060 FAR=0.009 FRR=0.287 ACC=0.985 ERR=0.148

[INFO] Training CAE for s034
tau=0.069135 FAR=0.001 FRR=0.988 ACC=0.980 ERR=0.494

[INFO] Training CAE for s035
tau=0.041281 FAR=0.000 FRR=0.988 ACC=0.981 ERR=0.494

[INFO] Training CAE for s036
tau=1.558007 FAR=0.002 FRR=0.037 ACC=0.998 ERR=0.019

[INFO] Training CAE for s037
tau=0.055014 FAR=0.001 FRR=0.988 ACC=0.980 ERR=0.494

[INFO] Training CAE for s038
tau=0.080635 FAR=0.000 FRR=0.963 ACC=0.981 ERR=0.481

[INFO] Training CAE for s039
tau=0.075753 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s040
tau=0.038266 FAR=0.000 FRR=0.988 ACC=0.981 ERR=0.494

[INFO] Training CAE for s041
tau=0.309553 FAR=0.011 FRR=0.438 ACC=0.980 ERR=0.225

[INFO] Training CAE for s042
tau=0.028331 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s043
tau=0.094458 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s044
tau=0.037255 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s046
tau=0.056261 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s047
tau=0.039988 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s048
tau=0.039588 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s049
tau=0.227962 FAR=0.009 FRR=0.475 ACC=0.982 ERR=0.242

[INFO] Training CAE for s050
tau=0.053505 FAR=0.001 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s051
tau=0.033043 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s052
tau=0.057765 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s053
tau=0.101121 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s054
tau=0.051329 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s055
tau=0.075349 FAR=0.000 FRR=0.975 ACC=0.981 ERR=0.487

[INFO] Training CAE for s056
tau=0.030998 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Training CAE for s057
tau=0.048016 FAR=0.000 FRR=1.000 ACC=0.980 ERR=0.500

[INFO] Wrote CAE baseline outputs:
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-
```

D.2 Keyboard SVM training and evaluation

Figure D.3 – Keyboard SVM final evaluation output showing explicit train/validation/test files, selected hyperparameters $C=20.0$, $\gamma=0.1$, and macro FAR/FRR/ACC.

```
hitesh@hitesh-2: Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments % python Apps/behavior/eval_kb_svm.py
[INFO] eval_kb_svm (explicit split) starting
[INFO] Train file: /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Data/keyboard_train_windows.csv
[INFO] Val file: /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Data/keyboard_val_windows.csv
[INFO] Test file: /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Data/keyboard_test_windows.csv
[INFO] Users retained: 51
[INFO] Train size: 2240, Val size: 400, Test size: 400
[INFO] Tuning SVM hyperparameters...
C=1.0 gamma=scale -> val_acc=0.5321
C=5.0 gamma=scale -> val_acc=0.5713
C=10.0 gamma=scale -> val_acc=0.5836
C=20.0 gamma=scale -> val_acc=0.5951
C=10.0 gamma=0.1 -> val_acc=0.5914
C=20.0 gamma=0.1 -> val_acc=0.6002
[INFO] Best hyperparams: C=20.0, gamma=0.1, val_acc=0.6002
[INFO] Training final SVM with best hyperparams on train+val...
[INFO] Final SVM training done
#002 FAR=0.011 FRR=0.537 ACC=0.979 n_g=80 n_i=4000
#003 FAR=0.008 FRR=0.325 ACC=0.984 n_g=80 n_i=4000
#004 FAR=0.017 FRR=0.438 ACC=0.975 n_g=80 n_i=4000
#005 FAR=0.011 FRR=0.380 ACC=0.983 n_g=80 n_i=4000
#007 FAR=0.010 FRR=0.455 ACC=0.978 n_g=80 n_i=4000
#008 FAR=0.011 FRR=0.500 ACC=0.980 n_g=80 n_i=4000
#010 FAR=0.011 FRR=0.275 ACC=0.984 n_g=80 n_i=4000
#011 FAR=0.011 FRR=0.425 ACC=0.983 n_g=80 n_i=4000
#012 FAR=0.005 FRR=0.212 ACC=0.990 n_g=80 n_i=4000
#013 FAR=0.013 FRR=0.388 ACC=0.980 n_g=80 n_i=4000
#015 FAR=0.012 FRR=0.500 ACC=0.979 n_g=80 n_i=4000
#016 FAR=0.007 FRR=0.380 ACC=0.988 n_g=80 n_i=4000
#017 FAR=0.002 FRR=0.138 ACC=0.995 n_g=80 n_i=4000
#018 FAR=0.013 FRR=0.542 ACC=0.970 n_g=80 n_i=4000
#019 FAR=0.006 FRR=0.200 ACC=0.990 n_g=80 n_i=4000
#020 FAR=0.009 FRR=0.637 ACC=0.979 n_g=80 n_i=4000
#021 FAR=0.009 FRR=0.450 ACC=0.980 n_g=80 n_i=4000
#022 FAR=0.001 FRR=0.050 ACC=0.999 n_g=80 n_i=4000
#024 FAR=0.008 FRR=0.263 ACC=0.987 n_g=80 n_i=4000
#025 FAR=0.006 FRR=0.125 ACC=0.992 n_g=80 n_i=4000
#026 FAR=0.010 FRR=0.237 ACC=0.986 n_g=80 n_i=4000
#027 FAR=0.010 FRR=0.375 ACC=0.983 n_g=80 n_i=4000
#028 FAR=0.004 FRR=0.100 ACC=0.994 n_g=80 n_i=4000
#029 FAR=0.016 FRR=0.637 ACC=0.972 n_g=80 n_i=4000
#030 FAR=0.008 FRR=0.400 ACC=0.984 n_g=80 n_i=4000
#031 FAR=0.013 FRR=0.662 ACC=0.975 n_g=80 n_i=4000
#032 FAR=0.011 FRR=0.762 ACC=0.974 n_g=80 n_i=4000
#033 FAR=0.004 FRR=0.200 ACC=0.992 n_g=80 n_i=4000
#034 FAR=0.011 FRR=0.588 ACC=0.977 n_g=80 n_i=4000
#035 FAR=0.009 FRR=0.588 ACC=0.979 n_g=80 n_i=4000
#036 FAR=0.000 FRR=0.037 ACC=0.999 n_g=80 n_i=4000
#037 FAR=0.000 FRR=0.600 ACC=0.970 n_g=80 n_i=4000
#038 FAR=0.009 FRR=0.325 ACC=0.985 n_g=80 n_i=4000
#039 FAR=0.005 FRR=0.212 ACC=0.991 n_g=80 n_i=4000
#040 FAR=0.005 FRR=0.325 ACC=0.980 n_g=80 n_i=4000
#041 FAR=0.004 FRR=0.225 ACC=0.991 n_g=80 n_i=4000
#042 FAR=0.003 FRR=0.113 ACC=0.995 n_g=80 n_i=4000
#043 FAR=0.004 FRR=0.007 ACC=0.994 n_g=80 n_i=4000
#044 FAR=0.009 FRR=0.450 ACC=0.983 n_g=80 n_i=4000
#046 FAR=0.009 FRR=0.725 ACC=0.976 n_g=80 n_i=4000
#047 FAR=0.010 FRR=0.700 ACC=0.976 n_g=80 n_i=4000
#048 FAR=0.012 FRR=0.613 ACC=0.976 n_g=80 n_i=4000
#049 FAR=0.009 FRR=0.175 ACC=0.988 n_g=80 n_i=4000
#050 FAR=0.009 FRR=0.600 ACC=0.979 n_g=80 n_i=4000
#051 FAR=0.005 FRR=0.738 ACC=0.980 n_g=80 n_i=4000
#052 FAR=0.001 FRR=0.062 ACC=0.998 n_g=80 n_i=4000
#053 FAR=0.006 FRR=0.312 ACC=0.980 n_g=80 n_i=4000
#054 FAR=0.005 FRR=0.502 ACC=0.984 n_g=80 n_i=4000
#055 FAR=0.004 FRR=0.350 ACC=0.989 n_g=80 n_i=4000
#056 FAR=0.005 FRR=0.725 ACC=0.981 n_g=80 n_i=4000
#057 FAR=0.007 FRR=0.588 ACC=0.981 n_g=80 n_i=4000
MACRO FAR=0.008 FRR=0.400 ACC=0.984
WRITE SVM metrics CSV -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/kb_svm_eval_metrics.csv
WRITE SVM macro JSON -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/kb_svm_eval_overview.json
WRITE SVM model + scaler -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-hitesh@hitesh-2 hitesh@hitesh-2 Cohlitesh@hitesh-2hitesh@hitesh-2
```


D.3 Mouse SVM training and evaluation

Figure D.4 – Mouse SVM training output showing grouped cross-validation, selected SVM settings and saved model artefacts.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments % python Apps/behavior/train_mouse_svm.py
[INFO] Loaded 23358 tuning windows for 10 users from /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Data/mouse_windows_train.csv
[INFO] Loaded 23358 final-train windows for 10 users from /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Data/mouse_windows_train_augmented.csv
[INFO] Unique tuning files: 65
[INFO] Tuning SVM hyperparameters with grouped CV by raw file...
C=50.0 gamma=0.1 -> cv_mean_acc=0.3256, cv_mean_macro_f1=0.2670
C=50.0 gamma=0.2 -> cv_mean_acc=0.2998, cv_mean_macro_f1=0.2429
C=50.0 gamma=0.3 -> cv_mean_acc=0.2909, cv_mean_macro_f1=0.2332
C=80.0 gamma=0.1 -> cv_mean_acc=0.3175, cv_mean_macro_f1=0.2602
C=80.0 gamma=0.2 -> cv_mean_acc=0.2966, cv_mean_macro_f1=0.2387
C=80.0 gamma=0.3 -> cv_mean_acc=0.2906, cv_mean_macro_f1=0.2328
C=100.0 gamma=0.1 -> cv_mean_acc=0.3158, cv_mean_macro_f1=0.2580
C=100.0 gamma=0.2 -> cv_mean_acc=0.2977, cv_mean_macro_f1=0.2392
C=100.0 gamma=0.3 -> cv_mean_acc=0.2923, cv_mean_macro_f1=0.2346
[INFO] Best hyperparams: C=50.0, gamma=0.1, cv_mean_acc=0.3256, cv_mean_macro_f1=0.2670
[INFO] Training final mouse SVM on augmented full train set...
[INFO] Final mouse SVM training done.

[INFO] Classification report on base-train data (sanity only):
      precision    recall  f1-score   support

 user12      0.83      0.78      0.80      4044
 user15      0.95      0.87      0.91      2594
 user16      0.89      0.73      0.80      3840
 user20      0.73      0.72      0.73      1730
 user21      0.80      0.89      0.84      2062
 user23      0.69      0.79      0.74      2088
 user29      0.74      0.80      0.77      1918
 user35      0.76      0.74      0.75      1684
 user7       0.83      0.88      0.85      2204
 user9       0.69      0.90      0.78      1194

 accuracy          0.80      0.80      0.80      23358
 macro avg      0.79      0.81      0.80      23358
 weighted avg   0.81      0.80      0.80      23358

SAVED:
 scaler -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Models/mouse/
 mouse_scaler.joblib
 model -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Models/mouse/
 mouse_svm.joblib
 meta -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/Models/mouse/
 mouse_meta.json
 tuning -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/mou
 se/mouse_tuning_results.csv
 summary -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/mo
 use/mouse_tuning_summary.json
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
```

Figure D.5 – Mouse SVM held-out evaluation output showing per-user FAR/FRR/ACC and saved mouse evaluation artefacts.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments % python Apps/behavior/eval_mouse.py
[INFO] eval_mouse (SVM) starting
[INFO] Loading test data from /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environm
ents/Data/mouse_windows_test.csv
[INFO] Evaluating 10 users
user12 FAR=0.160 FRR=0.662 ACC=0.589 n_g=3838 n_i=19954
user15 FAR=0.122 FRR=0.754 ACC=0.562 n_g=3630 n_i=20162
user16 FAR=0.110 FRR=0.688 ACC=0.601 n_g=3136 n_i=20656
user20 FAR=0.044 FRR=0.816 ACC=0.570 n_g=1508 n_i=22284
user21 FAR=0.111 FRR=0.660 ACC=0.615 n_g=1716 n_i=22076
user23 FAR=0.076 FRR=0.824 ACC=0.550 n_g=2016 n_i=21776
user29 FAR=0.068 FRR=0.778 ACC=0.577 n_g=1568 n_i=22224
user35 FAR=0.057 FRR=0.912 ACC=0.515 n_g=2574 n_i=21218
user7 FAR=0.046 FRR=0.598 ACC=0.678 n_g=2164 n_i=21628
user9 FAR=0.043 FRR=0.811 ACC=0.573 n_g=1642 n_i=22150

MACRO FAR=0.084 FRR=0.750 ACC=0.583

WROTE mouse SVM metrics CSV -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environ
ments/artifacts/mouse_svm_eval_metrics.csv
WROTE mouse SVM macro JSON -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environ
ments/artifacts/mouse_svm_eval_overview.json
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
```

D.4 Runtime plots and takeover replay

Figure D.6 – Real-time plot script output showing time-series, histogram and action-count plots saved under artifacts/realtime/.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments]
% python Apps/behavior/eval_realtime_plots.py
[OK] Wrote time-series plot -> artifacts/realtime/fig_trust_timeseries.png
[OK] Wrote histogram plot -> artifacts/realtime/fig_trust_histograms.png
[OK] Wrote action bar plot -> artifacts/realtime/fig_actions_bar.png
[OK] Wrote trust summary CSV -> artifacts/realtime/trust_summary.csv
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments
% █
```

Figure D.7 – Takeover replay script output showing generation of takeover_sim_kb.csv and takeover_sim_kb.png.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments]
% python make_takeover_plot.py
Wrote: /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-
-Environments/artifacts/realtime/takeover_sim_kb.csv
Wrote: /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-
-Environments/artifacts/realtime/takeover_sim_kb.png
Users: u1(enrolled)=s002, u2(attacker)=s003
Feature count: 18
Min/Max trust_u1_smooth: 0.0000 / 0.6549
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments
% █
```

D.5 Labelled live-stream policy evaluation

Figure D.8 – Labelled live-stream evaluation output showing 75.6% live policy accuracy, 95.5% genuine allow rate and 56.2% takeover detection rate.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %]
python Apps/behavior/eval_live_labelled.py

Live labelled policy evaluation
      metric      value
live_policy_accuracy  0.755849
genuine_allow_rate    0.954988
genuine_false_alarm_rate 0.045012
takeover_detection_rate_STEP_UP_or_LOCK 0.562130
takeover_lock_rate    0.075740
takeover_stepup_rate  0.486391
      n_genuine_rows  822.000000
      n_takeover_rows  845.000000
      n_total_rows    1667.000000

Saved -> /Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-
-Environments/artifacts/realtime/live_labelled_policy_metrics.csv
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
█
```

D.6 baseline comparison evidence

Figure D.9 – Keyboard grouped cross-validation baseline comparison output for logistic regression and SVM.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments % python Apps/behavior/cv_keyboard_baselines.py
[INFO] Keyboard baseline / CV evaluation
[INFO] Development rows: 16320
[INFO] Held-out test rows: 4080
[INFO] Users retained: 51
[INFO] Unique development sessions: 8

[INFO] Running grouped CV for: logreg_baseline
fold=1 acc=0.5431 macro_f1=0.5257
fold=2 acc=0.4899 macro_f1=0.4720
fold=3 acc=0.3473 macro_f1=0.3229
fold=4 acc=0.4512 macro_f1=0.4322
fold=5 acc=0.5199 macro_f1=0.5069
[INFO] logreg_baseline: cv_mean_acc=0.4703, cv_mean_macro_f1=0.4520, heldout_test_acc=0.5164, heldout_test_macro_f1=0.5011

[INFO] Running grouped CV for: svm_rbf_final
fold=1 acc=0.5568 macro_f1=0.5428
fold=2 acc=0.5603 macro_f1=0.5495
fold=3 acc=0.3735 macro_f1=0.3578
fold=4 acc=0.5207 macro_f1=0.5089
fold=5 acc=0.5324 macro_f1=0.5212
[INFO] svm_rbf_final: cv_mean_acc=0.5087, cv_mean_macro_f1=0.4961, heldout_test_acc=0.5998, heldout_test_macro_f1=0.5931

[INFO] Wrote outputs:
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/keyboard/keyboard_cv_folds.csv
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/keyboard/keyboard_cv_summary.csv
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/keyboard/keyboard_cv_summary.json
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
```

Figure D.10 – Mouse grouped cross-validation baseline comparison output for logistic regression and SVM.

```
[hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments % python Apps/behavior/cv_mouse_baselines.py
[INFO] Mouse baseline / CV evaluation
[INFO] Development rows: 23358
[INFO] Held-out test rows: 23792
[INFO] Users retained: 10
[INFO] Unique development files: 65

[INFO] Running grouped CV for: logreg_baseline
fold=1 acc=0.3368 macro_f1=0.2536
fold=2 acc=0.2887 macro_f1=0.2349
fold=3 acc=0.4325 macro_f1=0.3467
fold=4 acc=0.3072 macro_f1=0.2733
fold=5 acc=0.2767 macro_f1=0.2337
[INFO] logreg_baseline: cv_mean_acc=0.3284, cv_mean_macro_f1=0.2684, heldout_test_acc=0.2382, heldout_test_macro_f1=0.2294

[INFO] Running grouped CV for: svm_rbf_final
fold=1 acc=0.3406 macro_f1=0.2411
fold=2 acc=0.2488 macro_f1=0.2045
fold=3 acc=0.3214 macro_f1=0.2852
fold=4 acc=0.2995 macro_f1=0.2358
fold=5 acc=0.2514 macro_f1=0.2063
[INFO] svm_rbf_final: cv_mean_acc=0.2923, cv_mean_macro_f1=0.2346, heldout_test_acc=0.2462, heldout_test_macro_f1=0.2250

[INFO] Wrote outputs:
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/mouse/mouse_cv_folds.csv
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/mouse/mouse_cv_summary.csv
/Users/hitesh/Desktop/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments/artifacts/mouse/mouse_cv_summary.json
hitesh@Hitesh-2 Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments %
```


APPENDIX E AI USE AND CODE PROVENANCE TABLE

Table E.1 summarises where AI tools were used as support during coding, debugging, report writing, citation checking, diagram preparation and appendix preparation. The final code, results and interpretation were checked against the project files and outputs before submission.

Component	What I wrote / implemented	What AI helped with	What I changed after AI help	Where in repo / report
Browser monitor UI	I implemented the monitor page structure, event capture flow, trust display, action display, and connection to Django endpoints.	ChatGPT helped with HTML/JavaScript structure suggestions, fetch() request patterns, debugging stuck UI updates, and wording for explaining the client-server flow.	I adapted the JavaScript to my endpoint names, added in-flight request control, skipped idle/empty windows, and tested that live trust values updated correctly.	templates/behavior/monitor.html
Keyboard feature extraction	I defined the final 18-feature keyboard vector using dwell, down-down, up-down, backspace, burst and idle features.	ChatGPT helped with wording and small code examples for keydown/keyup handling, dwell timing, latency features, percentile logic, and edge cases.	I kept the final feature order consistent with the scaler/model, removed text-content dependency, and checked that runtime input matched the trained SVM input dimension.	Apps/behavior/ae_conditional.py; templates/behavior/monitor.html
Keyboard CAE baseline	I trained and evaluated the per-user autoencoder baseline using the prepared keyboard train/validation/test splits.	ChatGPT helped explain the autoencoder structure, threshold selection, feature-dimension errors, and CAE evaluation wording.	I corrected the evaluation to use validation data for threshold selection and held-out test data for final reporting, then regenerated CAE metrics and plots.	Apps/behavior/train_kb_cae_cmu.py; Apps/behavior/eval_kb_cae.py; Models/kb_cae; artifacts/
Keyboard SVM final model	I used the final keyboard SVM as the runtime keyboard scorer and reported verification-style FAR/FRR/ACC.	ChatGPT helped with sklearn syntax, model/scaler saving patterns, margin-to-trust explanation, and debugging evaluation output/plot generation.	I changed the final process to explicit train/validation/test files, selected $C = 20.0$ and $\gamma = 0.1$, saved the scaler/model, and checked that the live monitor reused the same scaler.	Apps/behavior/eval_kb_svm.py; Apps/behavior/ae_conditional.py; Models/kb_svm; artifacts/kb_svm_eval_*

Component	What I wrote / implemented	What AI helped with	What I changed after AI help	Where in repo / report
Mouse feature extraction	I used the final 23-feature mouse vector based on duration, point count, path length, speed, acceleration, jerk, displacement, bounding box, direction changes and pause ratio.	ChatGPT helped with wording the mouse feature definitions and explaining how the vector is used in training and runtime.	I ensured the feature order matched mouse_meta.json, and I made clear that the exact 23-feature vector is my implementation choice, not copied from a paper.	Apps/behavior/mouse_ingest.py; Models/mouse/mouse_meta.json
Mouse SVM training/evaluation	I trained the mouse SVM, saved the model/scaler/meta files, and evaluated it on held-out mouse windows.	ChatGPT helped with sklearn SVC patterns, grouped cross-validation explanation, metric table wording, and plotting/debugging assistance.	I used grouped cross-validation by raw file, selected C = 50.0 and gamma = 0.1, saved the final artefacts, and reported held-out FAR/FRR/ACC honestly as a support signal.	Apps/behavior/train_mouse_svm.py; Apps/behavior/eval_mouse.py; Models/mouse/; artifacts/mouse*
Face scoring pipeline	I implemented face as a lightweight support signal rather than a fully trained face-recognition or anti-spoofing model.	ChatGPT helped with wording the limits of detection/alignment, liveness, spoofing risk and why face is only support.	I removed overclaims about strong anti-spoofing, removed unverified references, and kept face as optional match/liveness support only.	Apps/behavior/face_runtime.py; Apps/behavior/views.py
Fusion and policy logic	I implemented fixed score-level fusion, clamp rules, smoothing/holding, and ALLOW / STEP_UP / LOCK thresholds.	ChatGPT helped phrase the fusion equations, explain that the weights are implementation choices, and debug wording around the policy logic.	I updated the final fusion to match the actual code: 0.8/0.2 behaviour fusion, 0.9/0.1 face trust, face support clamp, and thresholds 0.30 / 0.55.	Apps/behavior/trust_fusion.py; Apps/behavior/views.py
Django endpoints and server flow	I implemented the endpoint structure for streaming, scoring, fusion, face enrolment/scoring and logging.	ChatGPT helped explain the difference between URL endpoints and files, and helped draft the client-server implementation section.	I corrected the report wording so endpoints are described as URL routes implemented in views.py, not as separate files.	Apps/behavior/views.py; Apps/behavior/urls.py
Real-time logging and plots	I logged trust values and actions to CSV and generated trust time-series, histograms, action counts and takeover plots.	ChatGPT helped with matplotlib plotting patterns, file path debugging, and how to describe generated artefacts in Chapter 4.	I regenerated final plots from corrected live logs and used the final artefact paths in the report.	Apps/behavior/eval_realtime_plots.py; make_takeover_plot.py; artifacts/realtime/
Labelled live-stream evaluation	I created separate genuine and takeover live CSV recordings and evaluated live policy accuracy, genuine allow rate and takeover detection rate.	ChatGPT helped with policy-metric definitions and report wording for unseen live-data testing.	I reported the final live labelled result honestly: 75.6% policy accuracy, 95.5% genuine allow rate and 56.2% takeover detection.	Apps/behavior/eval_live_labelled.py; artifacts/realtime/live_labelled_policy_metrics.csv

Component	What I wrote / implemented	What AI helped with	What I changed after AI help	Where in repo / report
Report writing and restructuring	I wrote the technical content using my implementation, outputs and experiments.	ChatGPT helped improve structure, grammar, section ordering, wording clarity, and rewriting after marker feedback.	I checked the wording against my actual code/results, removed unsupported claims, corrected citations, and rewrote sections in my own style before final use.	Dissertation Chapters 1–6
Literature review citation cleanup	I selected the final papers and used them to support the final report claims.	ChatGPT helped identify risky citation wording and suggested safer wording where claims were too broad.	I removed claims that were not directly supported, removed/corrected bad references, and separated literature claims from my own implementation choices.	Chapter 2 and References
Diagrams and visual material	I defined the system components, dataflow and workflow shown in the diagrams.	Napkin AI helped with diagram layout/styling, and ChatGPT helped with captions and explanation wording.	I checked that the diagrams matched my actual architecture and labelled them as design/supporting visuals, not experimental results.	Appendix A

Table E.1 – AI Use and Code Provenance

Note: I did not treat AI output as final without checking it. Where ChatGPT helped with code, debugging or report wording, I adapted the suggestions, ran the scripts myself, checked the outputs, and corrected the report so that it matched the final code and artefacts. The final results, interpretation, screenshots and submitted implementation are based on experiments I ran and outputs generated from my project files.

APPENDIX F AI PROMPT/OUTPUT SCREENSHOTS

Figure F.1 – Keyboard margin-to-trust helper

Prompt asked: I asked for a small Python helper that converts a positive SVM margin into a trust score between 0 and 1.

AI helped with: ChatGPT suggested the logistic conversion structure and handling invalid or non-positive margins.

My use: I used this idea in the keyboard runtime scorer and checked that it matched the live trust behaviour.

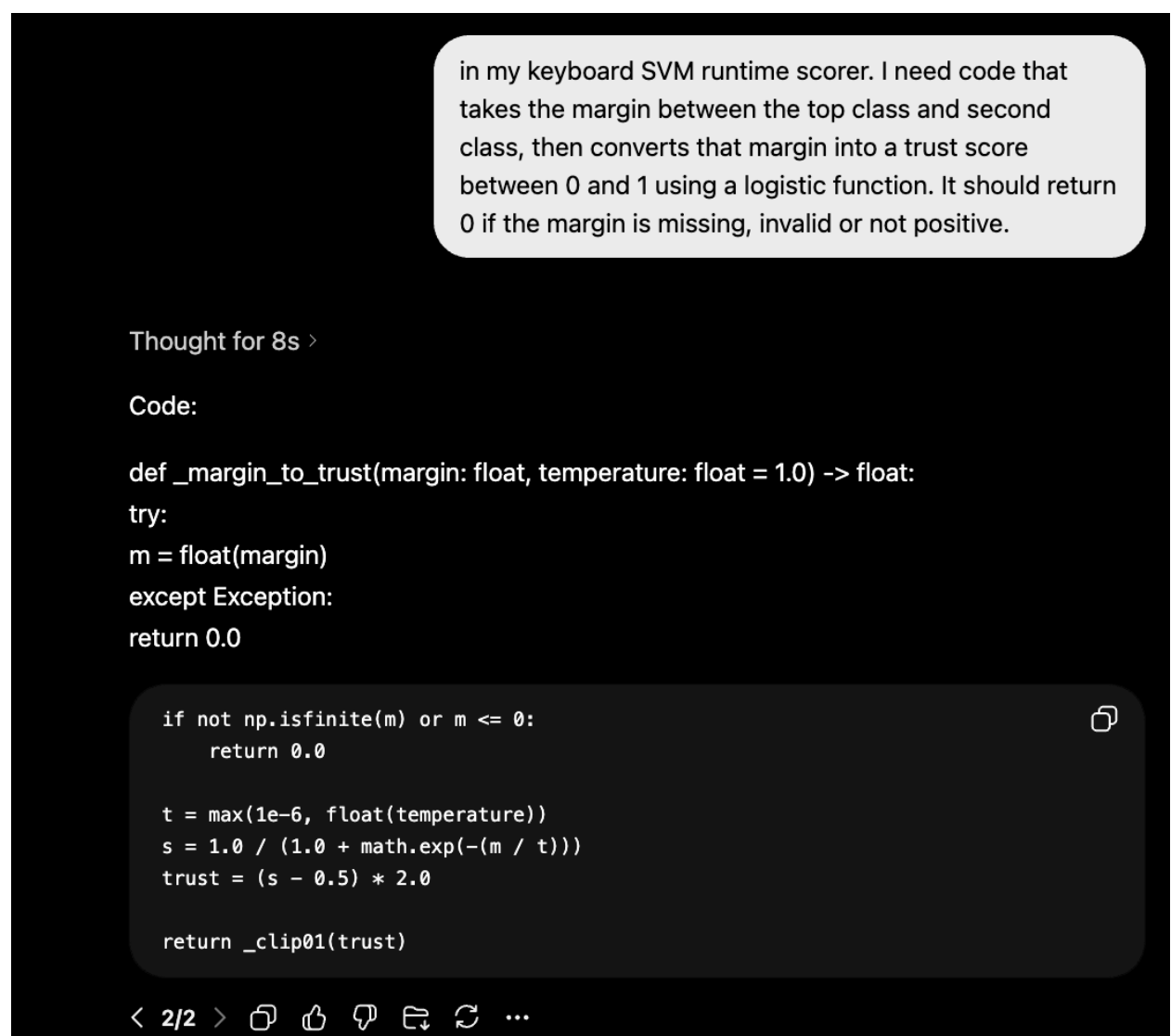


Figure F.2 – Keyboard top-two score margin

Prompt asked: I asked how to take SVM decision scores, find the top two scores and calculate the margin.

AI helped with: ChatGPT gave a simple code pattern for sorting scores and converting the top-two gap into trust.

My use: I adapted this into the keyboard SVM runtime trust logic instead of using raw maximum confidence.

I am working on a Python runtime scorer for keyboard SVM trust. Give me only the Python code for this logic that take SVM decision scores, get the top score and second top score, calculate the margin, then convert that margin into a trust value between 0 and 1 using a logistic function. Keep it simple and return the trust value. "Code:"



Thought for a few seconds >

Code:

```
scores = np.asarray(scores, dtype="float32").ravel()
```

```
if scores.size == 0:
```

```
    trust = 0.0
```

```
else:
```

```
    sorted_scores = np.sort(scores)
```

```
    top_score = float(sorted_scores[-1])
```

```
    if sorted_scores.size >= 2:
        second_score = float(sorted_scores[-2])
        margin = top_score - second_score
    else:
        margin = top_score
```



```
    trust = 1.0 / (1.0 + np.exp(-margin))
    trust = float(max(0.0, min(1.0, trust)))
```

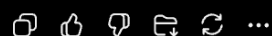


Figure F.3 – Keyboard 18-feature vector

Prompt asked: I asked for code to build the final 18 keyboard features from `keydown` and `keyup` events.

AI helped with: ChatGPT suggested structure for dwell, down-down, up-down, backspace, burst and idle features.

My use: I kept the feature order consistent with my trained scaler/model and tested it in the browser monitor.

my keyboard feature extraction code I need JavaScript code that builds my final 18 keyboard features from recent keydown and keyup events. The features should include dwell timing, down-down gaps, up-down gaps, backspace rate, burst mean and idle fraction. Keep it simple and use helper functions like mean, std and pct if they already exist.

Thought for 4s >

Code:

```
function buildKbFeatures() {
  const now = performance.now();
  const winMs = 3000;
  const ev = keyEvents.filter(e => now - e.t <= winMs).sort((a, b) => a.t - b.t);

  const downEvents = ev.filter(e => e.type === 'down');
  if (!downEvents.length) return null;

  const dwellMapLocal = new Map();
  const dwellSec = [];

  for (const e of ev) {
    if (e.type === 'down') {
      dwellMapLocal.set(e.key, e.t);
    } else if (e.type === 'up') {
      const t0 = dwellMapLocal.get(e.key);
      if (typeof t0 === 'number' && e.t >= t0) {
        dwellSec.push((e.t - t0) / 1000.0);
      }
      dwellMapLocal.delete(e.key);
    }
  }

  const downTimes = downEvents.map(e => e.t).sort((a, b) => a - b);
  const upTimes = ev.filter(e => e.type === 'up').map(e => e.t).sort((a, b) => a - b);

  const ddSec = [];
  for (let i = 1; i < downTimes.length; i++) {
    ddSec.push((downTimes[i] - downTimes[i - 1]) / 1000.0);
  }

  const udSec = [];
  let j = 0;
  for (let i = 0; i < upTimes.length; i++) {
    while (j < downTimes.length && downTimes[j] < upTimes[i]) j++;
    if (j < downTimes.length) {
      udSec.push((downTimes[j] - upTimes[i]) / 1000.0);
    }
  }

  const backspaces = downEvents.filter(e => e.key === 'Backspace').length;
  const totalDowns = downEvents.length;
```

```
const backspaces = downEvents.filter(e => e.key === 'Backspace').length;
const totalDowns = downEvents.length;
const backspace_rate = totalDowns ? backspaces / totalDowns : 0;

function computeBurstMean(ddVals) {
  if (!ddVals.length) return 0;
  const flags = ddVals.map(v => v < 0.150);
  let run = 0;
  const runs = [];

  for (const f of flags) {
    if (f) {
      run += 1;
    } else if (run > 0) {
      runs.push(run);
      run = 0;
    }
  }

  if (run > 0) runs.push(run);
  return runs.length ? mean(runs) : 0;
}

const idle_frac = udSec.length ? (udSec.filter(v => v > 1.0).length / udSec.length) : 1.0;
const burst_mean = computeBurstMean(ddSec);

const feats18 = [
  mean(dwellSec), std(dwellSec), pct(dwellSec, 10), pct(dwellSec, 50), pct(dwellSec, 90),
  mean(ddSec), std(ddSec), pct(ddSec, 10), pct(ddSec, 50), pct(ddSec, 90),
  mean(udSec), std(udSec), pct(udSec, 10), pct(udSec, 50), pct(udSec, 90),
  backspace_rate,
  burst_mean,
  idle_frac
];

return feats18.map(v => (isFinite(v) ? Number(v) : 0));
}
```

📄 📌 🔍 🗑️ ↺ ⋮

Figure F.4 – Skipping empty keyboard and mouse windows

Prompt asked: I asked for frontend code that returns early when both keyboard and mouse features are missing.

AI helped with: ChatGPT suggested a simple idle-check block and waiting-status message.

My use: I used this to avoid sending empty or fake zero feature vectors to the backend during idle periods.

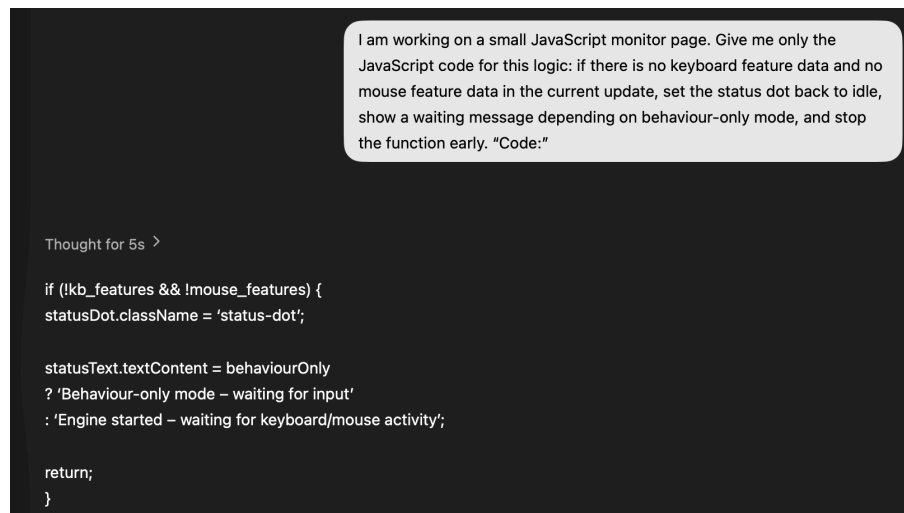


Figure F.5 – Face stale-score handling

Prompt asked: I asked for JavaScript logic to handle missing, invalid or `no_face` backend responses.

AI helped with: ChatGPT suggested clearing face match and liveness values and showing N/A in the UI.

My use: I used this to stop old face scores from staying active when the face was no longer detected.

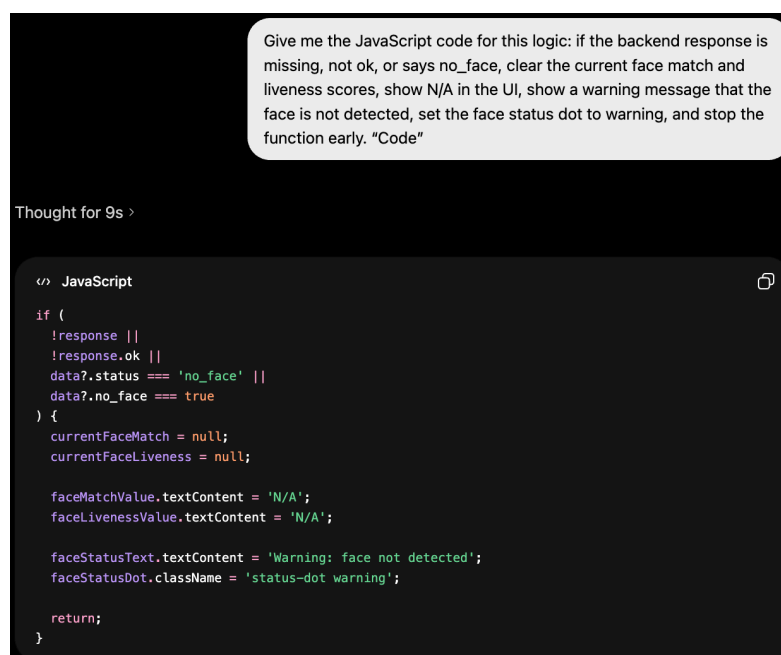


Figure F.6 – Django fusion input handling

Prompt asked: I asked for backend code to safely read keyboard, mouse and face trust values from the request body.

AI helped with: ChatGPT suggested converting values to float or `None` before passing them into fusion functions.

My use: I kept missing signals as `None` so the backend would not treat missing data as a zero-trust event.

Django endpoint for my project. Give me only the Python code for this logic: read kb_trust, mouse_trust, face_match and liveness from the request body, convert them safely to float or None, then call the trust fusion functions. If a value is missing, keep it as None instead of forcing it to zero. "Code:"

Thought for 8s >

Code:

```
kb_trust = _to_float_or_none(body.get("kb_trust"))  
  
mouse_trust = _to_float_or_none(body.get("mouse_trust"))  
  
face_match = _to_float_or_none(body.get("face_match"))  
  
liveness = _to_float_or_none(body.get("liveness"))  
  
behaviour_trust = trust_fusion.fuse_behaviour(kb_trust, mouse_trust)  
  
face_trust = trust_fusion.fuse_face(face_match, liveness)  
  
overall_trust = trust_fusion.fuse_overall(behaviour_trust, face_trust)  
  
action = trust_fusion.trust_policy_action(overall_trust)
```

Figure F.7 – Policy threshold logic

Prompt asked: I asked for simple Python policy code using the thresholds 0.30 and 0.55.

AI helped with: ChatGPT gave the basic LOCK, STEP_UP and ALLOW branching structure.

My use: I used this logic as the final prototype policy and reported the thresholds as implementation choices.

For the policy thresholds , I am working on a Python trust policy function. Give me the Python code for this logic: if trust is below 0.30 return LOCK, if trust is below 0.55 return STEP_UP, otherwise return ALLOW. "Code:"

Thought for a couple of seconds >

Code:

```
if trust < 0.30:  
    return "LOCK"  
elif trust < 0.55:  
    return "STEP_UP"  
else:  
    return "ALLOW"
```

Figure F.8 – Mouse probability-to-trust conversion

Prompt asked: I asked how to convert mouse SVM class probability into a trust score.

AI helped with: ChatGPT suggested removing the random class baseline, clamping the result and applying gamma scaling.

My use: I used this idea to make mouse trust more usable in the live monitor instead of using raw probability directly.

for svm , I need code that converts the maximum predicted class probability into a trust score. It should account for the random uniform baseline based on number of classes, clamp the result between 0 and 1, and then apply a small gamma boost so mid values are not too low.

Thought for a couple of seconds >

Code:

```
def proba_to_trust(pred_prob: float, n_classes: Optional[int] = None) -> float:
    try:
        p = float(pred_prob)
    except Exception:
        return 0.0
```

```
        if not np.isfinite(p) or p <= 0.0:
            return 0.0

        if n_classes and n_classes > 1:
            uniform = 1.0 / float(n_classes)
            if p <= uniform:
                t = 0.0
            else:
                t = (p - uniform) / (1.0 - uniform)
        else:
            t = p

        t = max(0.0, min(1.0, t))

        gamma = 0.3
        t = t ** gamma

        return float(max(0.0, min(1.0, t)))
```

Figure F.9 – Labelled live-stream evaluation logic

Prompt asked: I asked for code to evaluate genuine and takeover live CSV recordings using policy actions.

AI helped with: ChatGPT suggested counting ALLOW as correct for genuine rows and STEP_UP/LOCK as correct for takeover rows.

My use: I used this to report live policy accuracy, genuine allow rate and takeover detection rate.

i want a Python evaluation script for labelled live trust logs. Give me only the Python code for this logic: genuine rows are correct only when action is ALLOW, takeover rows are correct when action is STEP_UP or LOCK, then calculate live policy accuracy, genuine allow rate, false alarm rate, takeover detection rate, lock rate and step-up rate. "Code:"

Thought for 4s >

Code:

```
genuine = df[df["label"] == "genuine"].copy()
takeover = df[df["label"] == "takeover"].copy()

genuine_correct = genuine["action"].eq("ALLOW")
takeover_detected = takeover["action"].isin(["STEP_UP", "LOCK"])

n_genuine = len(genuine)
n_takeover = len(takeover)
n_total = n_genuine + n_takeover

genuine_allow_rate = genuine_correct.mean() if n_genuine else 0.0
genuine_false_alarm_rate = 1.0 - genuine_allow_rate if n_genuine else 0.0

takeover_detection_rate = takeover_detected.mean() if n_takeover else 0.0
takeover_lock_rate = takeover["action"].eq("LOCK").mean() if n_takeover else 0.0
takeover_stepup_rate = takeover["action"].eq("STEP_UP").mean() if n_takeover else 0.0

live_policy_accuracy = (
    (genuine_correct.sum() + takeover_detected.sum()) / n_total
    if n_total else 0.0
)

metrics = {
    "live_policy_accuracy": live_policy_accuracy,
    "genuine_allow_rate": genuine_allow_rate,
    "genuine_false_alarm_rate": genuine_false_alarm_rate,
    "takeover_detection_rate_STEP_UP_or_LOCK": takeover_detection_rate,
    "takeover_lock_rate": takeover_lock_rate,
    "takeover_stepup_rate": takeover_stepup_rate,
    "n_genuine_rows": n_genuine,
    "n_takeover_rows": n_takeover,
    "n_total_rows": n_total,
}
```

📄 📌 🔍 ⚙️ ↻️ ⋮

Figure F.10 – Loading saved keyboard SVM model and scaler

Image: Screenshot 2026-05-22 at 1.39.58 PM.png

This screenshot shows a prompt/output example for loading the saved keyboard SVM model and scaler from `Models/kb_svm`. It also shows reshaping one 18-feature keyboard vector, scaling it, checking the feature count, and getting SVM decision scores.

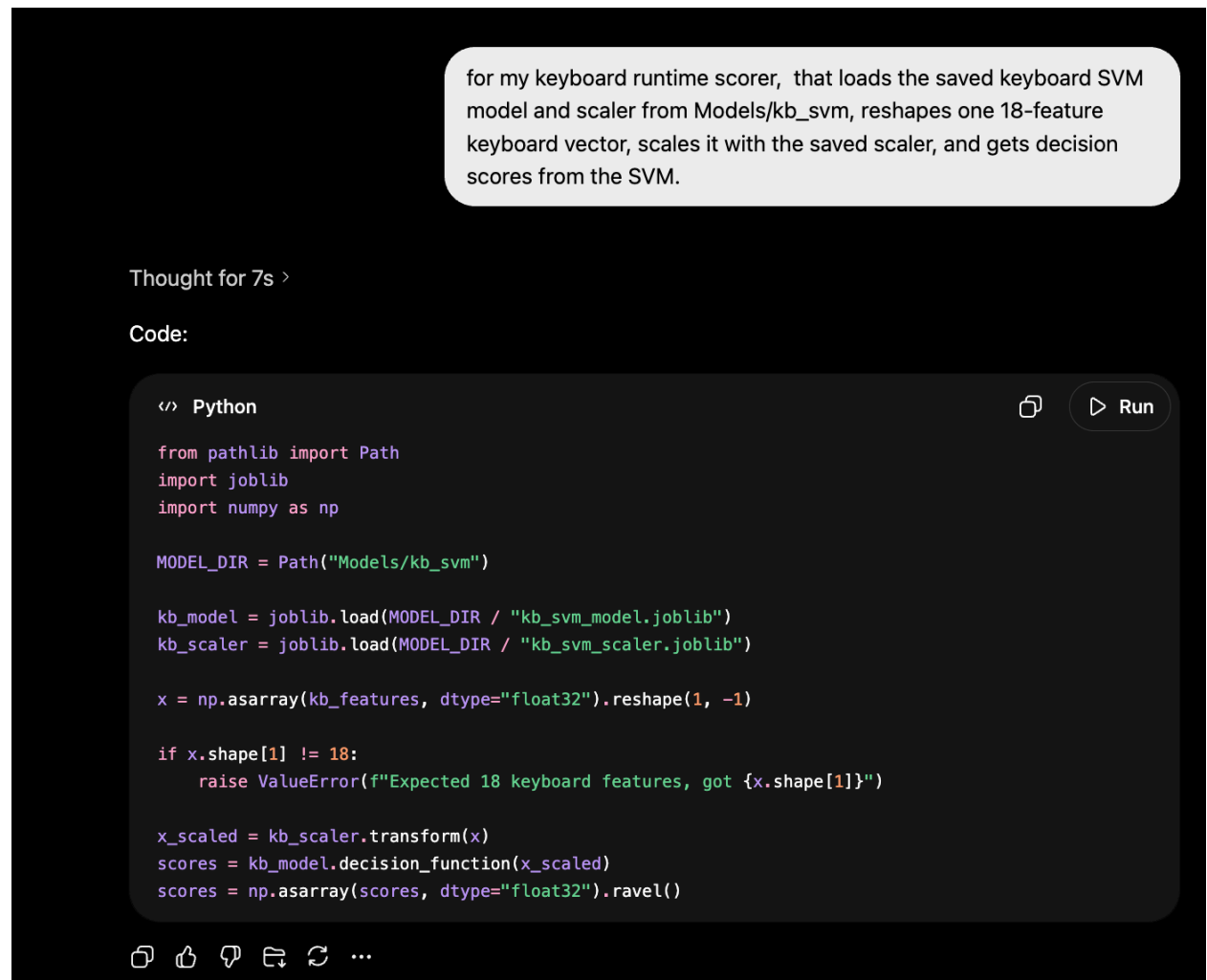


Figure F.11 – Keyboard SVM margin-to-trust calculation

Image: Screenshot 2026-05-22 at 1.38.52 PM.png

This screenshot shows a prompt/output example for using SVM decision scores to find the top and second-top scores. The margin is then converted into a trust value, which supports the keyboard runtime trust logic.

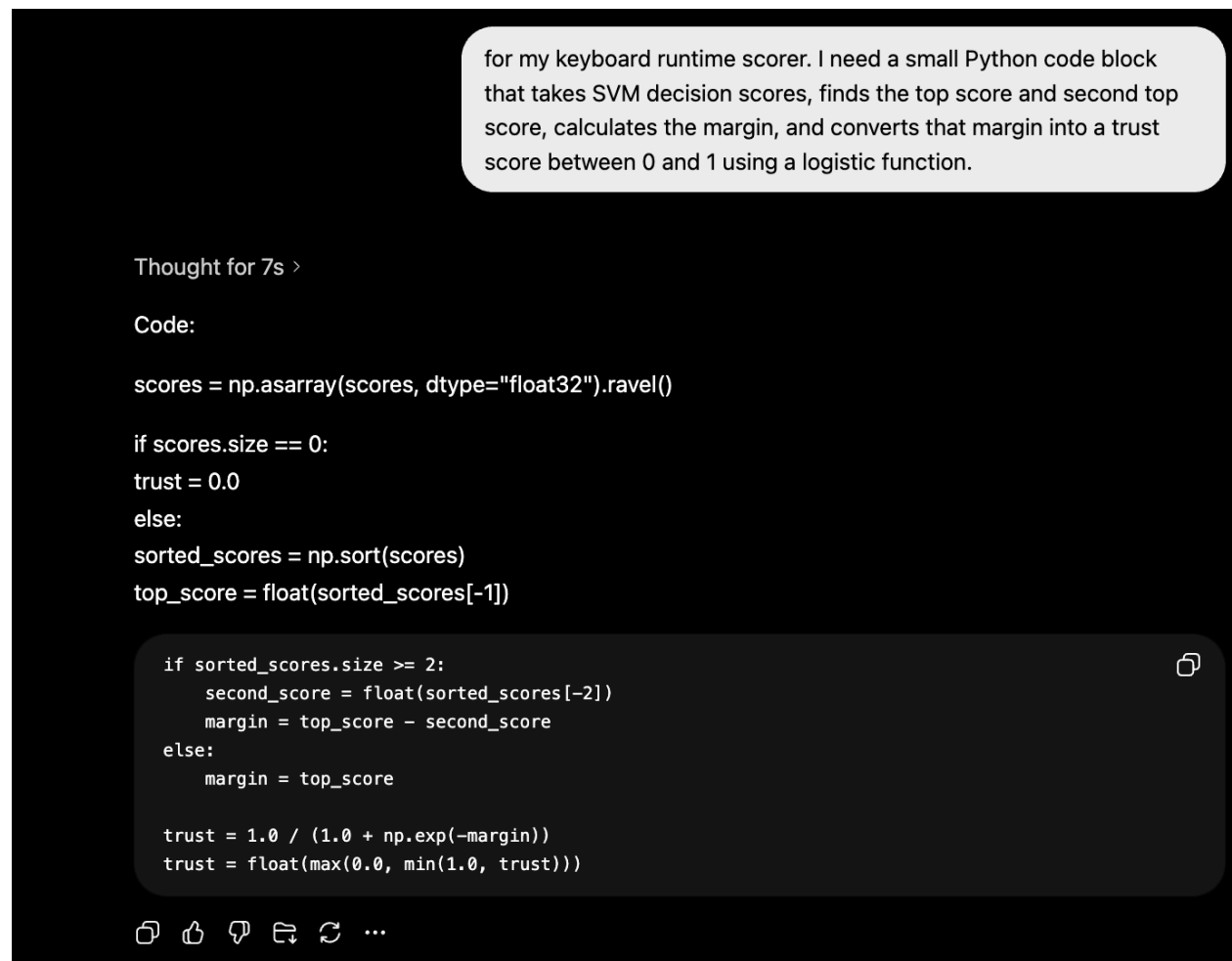


Figure F.12 – Labelled live policy evaluation script

Image: Screenshot 2026-05-22 at 1.42.27 PM.png

This screenshot shows a longer prompt/output example for loading genuine and takeover live CSV files. It calculates live policy accuracy, genuine allow rate, false alarm rate, takeover detection rate, lock rate and step-up rate.

on my labelled live evaluation script. that loads genuine and takeover live CSV files, labels them, combines them, marks ALLOW as correct for genuine rows, marks STEP_UP or LOCK as correct for takeover rows, calculates live policy accuracy and the main rates, and saves the metrics to a CSV.

Thought for 10s >

Code:

```
Python
import pandas as pd
from pathlib import Path

GENUINE_CSV = Path("Data/live_genuine.csv")
TAKEOVER_CSV = Path("Data/live_takeover.csv")
OUT_CSV = Path("artifacts/realtime/live_policy_metrics.csv")

genuine_df = pd.read_csv(GENUINE_CSV)
takeover_df = pd.read_csv(TAKEOVER_CSV)

genuine_df["label"] = "genuine"
takeover_df["label"] = "takeover"

df = pd.concat([genuine_df, takeover_df], ignore_index=True)

df["action"] = df["action"].astype(str).str.strip().str.upper()

genuine = df[df["label"] == "genuine"].copy()
takeover = df[df["label"] == "takeover"].copy()

genuine_correct = genuine["action"].eq("ALLOW")
takeover_detected = takeover["action"].isin(["STEP_UP", "LOCK"])

n_genuine = len(genuine)
n_takeover = len(takeover)
n_total = n_genuine + n_takeover

genuine_allow_rate = genuine_correct.mean() if n_genuine else 0.0
genuine_false_alarm_rate = 1.0 - genuine_allow_rate if n_genuine else 0.0

takeover_detection_rate = takeover_detected.mean() if n_takeover else 0.0
takeover_lock_rate = takeover["action"].eq("LOCK").mean() if n_takeover else 0.0
takeover_stepup_rate = takeover["action"].eq("STEP_UP").mean() if n_takeover else 0.0

live_policy_accuracy = (
    (genuine_correct.sum() + takeover_detected.sum()) / n_total
    if n_total else 0.0
)

metrics = {
    "live_policy_accuracy": live_policy_accuracy,
    "genuine_allow_rate": genuine_allow_rate,
    "genuine_false_alarm_rate": genuine_false_alarm_rate,
    "takeover_detection_rate_STEP_UP_or_LOCK": takeover_detection_rate,
    "takeover_lock_rate": takeover_lock_rate,
    "takeover_stepup_rate": takeover_stepup_rate,
    "n_genuine_rows": n_genuine,
    "n_takeover_rows": n_takeover,
    "n_total_rows": n_total,
}

OUT_CSV.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame([metrics]).to_csv(OUT_CSV, index=False)

print(metrics)
```

Figure F.13 – Keyboard SVM scaler fitting

Image: Screenshot 2026-05-22 at 1.37.54 PM.png

This screenshot shows a prompt/output example for fitting `StandardScaler` only on training features and then transforming validation and test features using the same scaler. This supports the correct training flow and avoids data leakage.

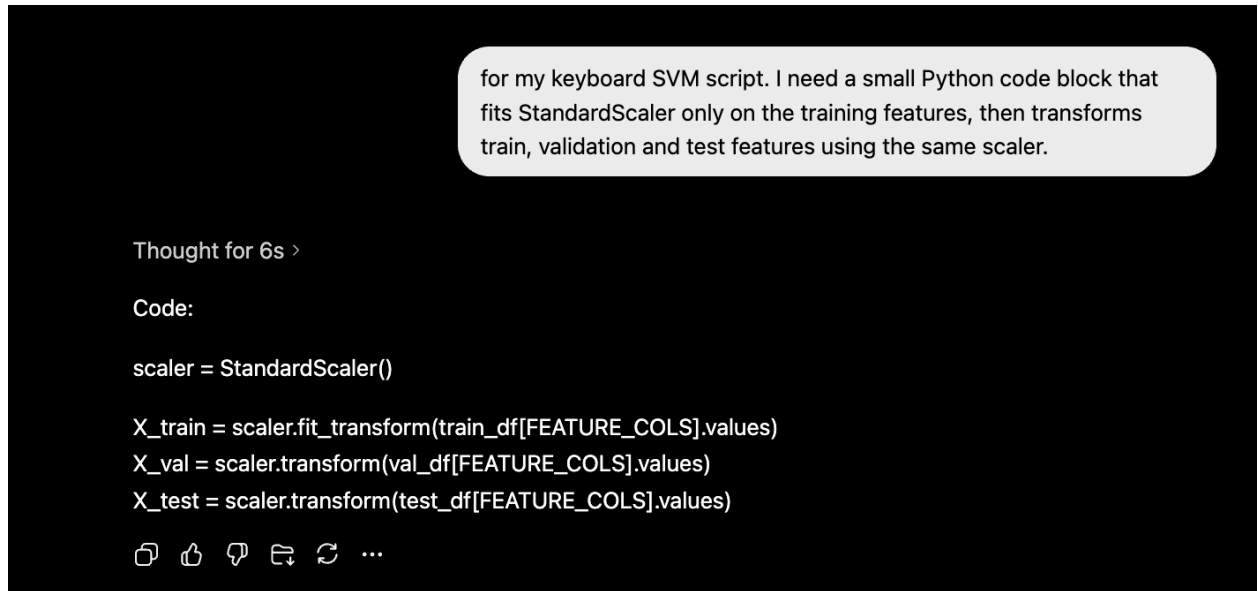


Figure F.14 – Keyboard CAE train/validation/test evaluation flow

Image: Screenshot 2026-05-22 at 1.32.46 PM.png

This screenshot shows a prompt/output example for training and evaluating a per-user autoencoder using prepared keyboard train, validation and test split files. It shows the idea of using validation data for threshold selection and held-out test data for final metrics.

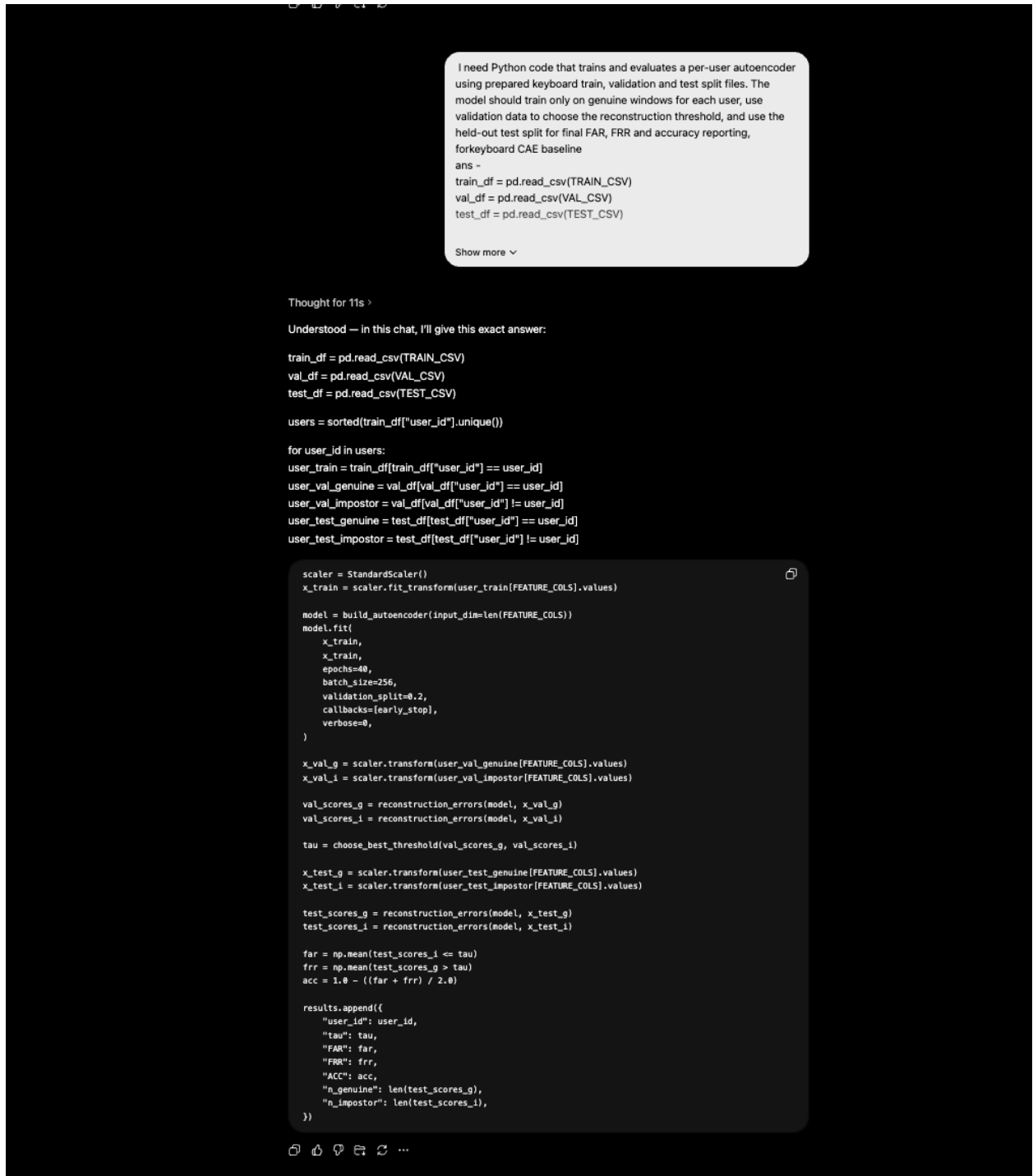
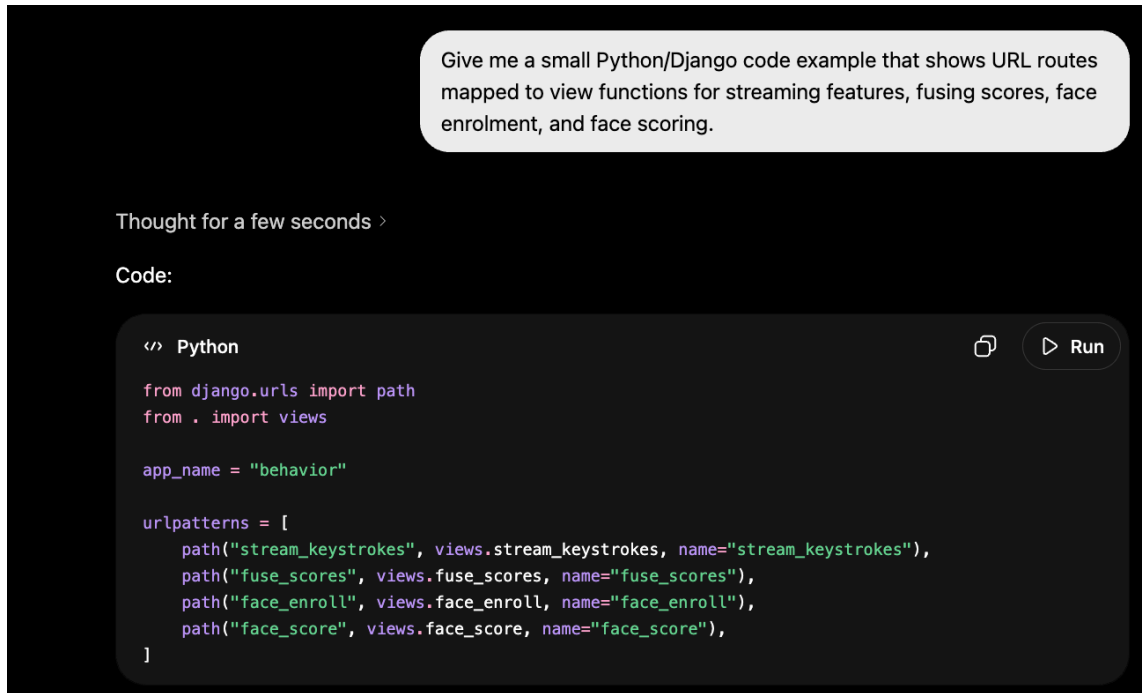


Figure F.15 – Django URL route mapping

Image: Screenshot 2026-05-22 at 1.46.54 PM.png

This screenshot shows a prompt/output example for mapping Django URL paths to view functions. It supports the explanation that `/stream_keystrokes`, `/fuse_scores`, `/face_enroll` and `/face_score` are URL endpoints, not separate files.



Give me a small Python/Django code example that shows URL routes mapped to view functions for streaming features, fusing scores, face enrolment, and face scoring.

Thought for a few seconds >

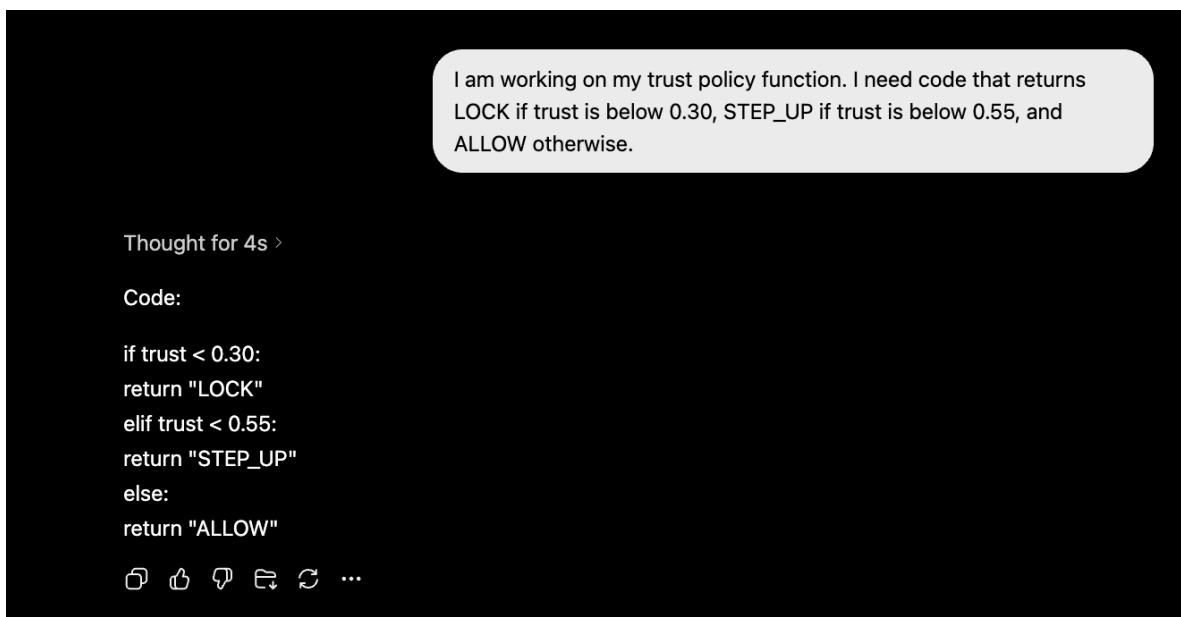
Code:

```
</> Python 📄 ▶ Run  
  
from django.urls import path  
from . import views  
  
app_name = "behavior"  
  
urlpatterns = [  
    path("stream_keystrokes", views.stream_keystrokes, name="stream_keystrokes"),  
    path("fuse_scores", views.fuse_scores, name="fuse_scores"),  
    path("face_enroll", views.face_enroll, name="face_enroll"),  
    path("face_score", views.face_score, name="face_score"),  
]
```

Figure F.16 – Trust policy threshold function

Image: Screenshot 2026-05-22 at 1.54.15 PM.png

This screenshot shows a prompt/output example for the ALLOW / STEP_UP / LOCK threshold logic. It returns LOCK below 0.30, STEP_UP below 0.55, and ALLOW otherwise.



I am working on my trust policy function. I need code that returns LOCK if trust is below 0.30, STEP_UP if trust is below 0.55, and ALLOW otherwise.

Thought for 4s >

Code:

```
if trust < 0.30:  
    return "LOCK"  
elif trust < 0.55:  
    return "STEP_UP"  
else:  
    return "ALLOW"
```

📄 📌 🔍 🗑️ ↺ ...

Figure F.17 – Real-time action-count plotting

Image: Screenshot 2026-05-22 at 2.01.20 PM.png

This screenshot shows a prompt/output example for generating an action-count bar chart from the live trust log. This supports the real-time plotting evidence used for policy decision counts.

```
in my realtime plot script. I need code that reads
live_trust_timeseries.csv and saves a trust time-series plot, trust
histogram plot and action-count bar chart.
plt.figure(figsize=(10, 5))
for c in trust_cols:
    vals = df[c].dropna()
    if not vals.empty:
        plt.hist(vals, bins=20, alpha=0.5, label=c)

plt.title("Trust Score Distributions")
plt.xlabel("Trust score")

Show more ▾
```

Thought for 4s >

Code:

```
</> Python 📄 ▶ Run

if "action" in df.columns:
    action_counts = (
        df["action"]
        .dropna()
        .astype(str)
        .str.strip()
        .str.upper()
        .value_counts()
    )

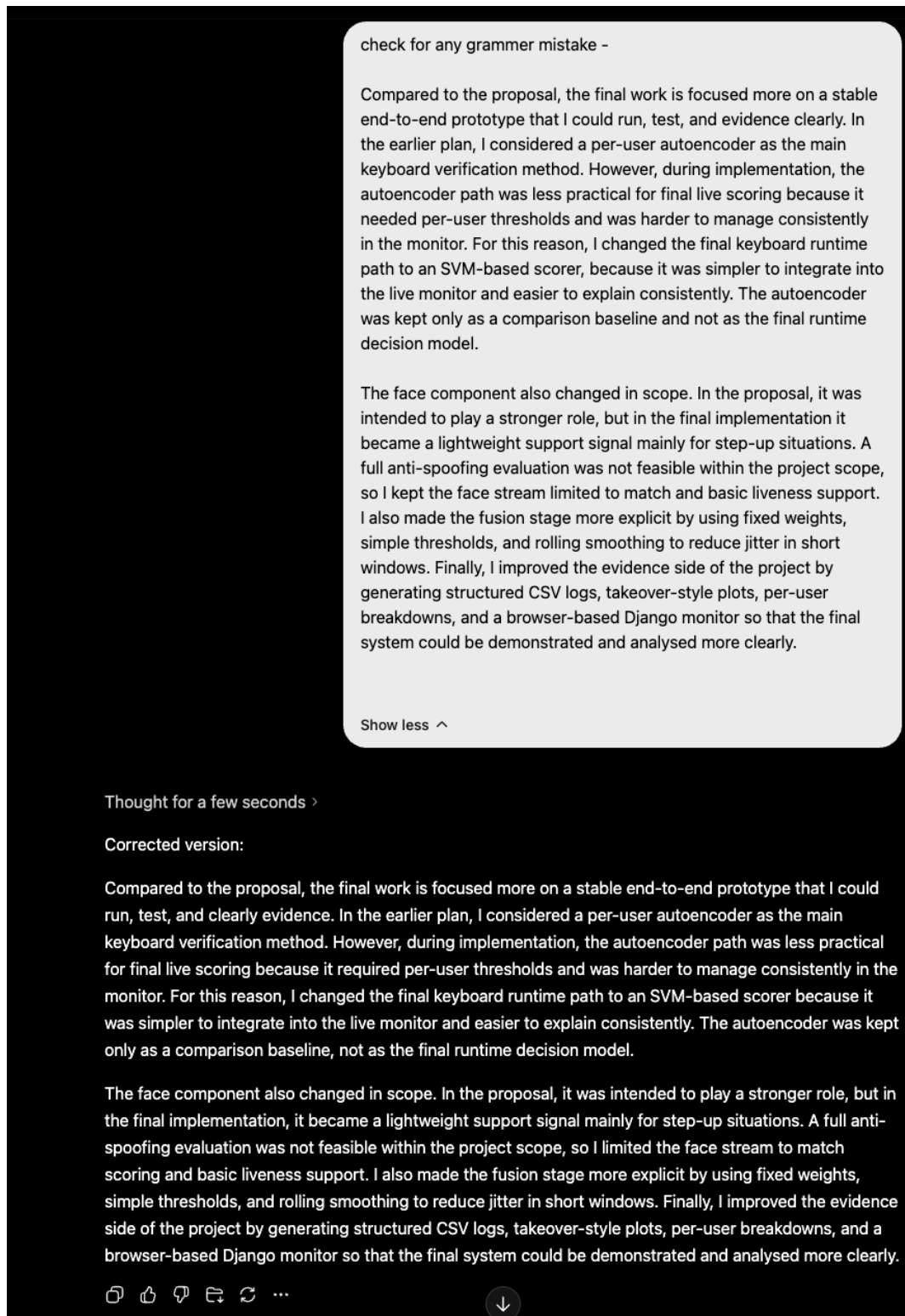
    if not action_counts.empty:
        plt.figure(figsize=(7, 4))
        action_counts.plot(kind="bar")

        plt.title("Policy Action Counts")
        plt.xlabel("Action")
        plt.ylabel("Count")
        plt.xticks(rotation=0)
        plt.tight_layout()
        plt.savefig(OUT_DIR / "fig_actions_bar.", dpi=200)
        plt.close()
```


Figure F.18 – Report grammar and wording refinement

Image: Screenshot 2026-05-22 at 2.04.20 PM.png

This screenshot shows a prompt/output example where report wording was checked and corrected. It supports the disclosure that ChatGPT was used for grammar, clarity and wording refinement, while keeping the implementation claims aligned with the final project.



APPENDIX G – CODE AND DATA REPOSITORIES

Project Source Code Repository

G.1 Project GitHub repository

URL: <https://github.com/Hitesh-m-Badgujar/Continuous-Authentication-to-Detect-Session-Hijacking-in-Remote-Work-Environments.git>

Main project directories for implementation, data, models and generated artefacts.

G.2 CMU-style keystroke dynamics dataset (Kaggle)

URL: <https://www.kaggle.com/datasets/carnegiecylab/keystroke-dynamics-benchmark-dataset>

G.3 BALABIT remote desktop / mouse dynamics dataset (Kaggle)

URL: <https://www.kaggle.com/datasets/gvkrishnakireeti/balabit-dataset>